VANIER
CÉGEP/COLLEGE

Project Planning & Prototyping

J. F. N. Salik,

247-519-VA

Department of Computer Engineering Technology
Vanier College
821 Sainte-Croix Avenue
Saint-Laurent, Québec H4L 3X9

This document was last modified on
Friday 29th May, 2026 at 5:26pm.

Title	**Project Planning & Prototyping**
Date	May 29, 2026
Author	J. F. N. Salik, (salikj@vaniercollege.qc.ca)
Identification	247-519-VA
Template	Sparkle Version 1.1 for LaTeX 2ε

VANIER
CÉGEP/COLLEGE

Contents

CHAPTER 1

OVERVIEW: PROTOTYPES, PRODUCTS & THE BUSINESS OF ENGINEERING

Every project begins with a claim: *this idea is worth building.* Your job in Project I is to find out whether that claim is true—cheaply, quickly, and with enough documentation that the answer is defensible. By the end of this course you will have built a working proof-of-concept, assembled the record that shows how you got there, and produced the business case that connects your hands-on work to the real world of engineering firms and markets. This chapter gives you the map. You will see what you are building, why it matters, what separates a prototype from a finished product, and what turns a validated prototype into revenue.

Learning Objectives

1. State what a prototype is for and describe the two deliverables you will produce in Project I.
2. Describe how a proven prototype becomes a product and identify the link between Project I and Project II.
3. Explain your role as a technician working alongside engineers.
4. Explain why industry work is justified by the value it creates, and interpret a negative economic finding correctly.
5. Identify the principal business models that turn a validated prototype into revenue.

1.1 What You Will Build

You will build two things: a **proof-of-concept prototype** and **the record of how you got there.**

The prototype is a working artifact that answers a specific technical and business question. It does not need to be pretty, manufacturable, or certifiable. It needs to

demonstrate that the core idea functions under realistic conditions.

The record is a portfolio of documents—specification, risk register, architecture diagrams, build logs, test data, and cost analysis—that accumulate as you work. You do not write the portfolio at the end. You assemble it from artifacts you produce at each stage. If you work that way, the portfolio is nearly complete before you realize you are making one.

Together, the prototype and its record constitute your output from Project I. The record is as important as the hardware: an engineer can reproduce, evaluate, and extend documented work; undocumented work is disposable.

1.2 The Course Map: Two Tracks Over Twelve Weeks

Figure 1.1 shows the full engineering journey from concept to market. Project I occupies the second stage—the prototype—and Project II the third, where the proven concept is developed into a manufacturable product. The arrow between them is not automatic: what crosses it is the documentation you create in Project I.

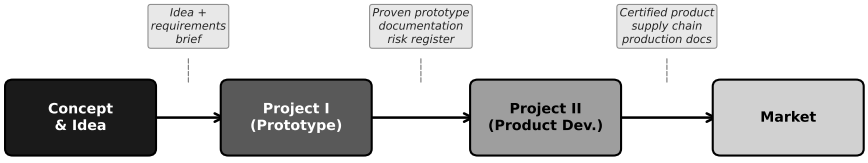

Figure 1.1: The engineering pipeline from concept to market. Project I delivers the proven prototype and its documentation; Project II turns that foundation into a manufacturable product.

The course runs two **parallel tracks**: a theory track and a lab (practice) track. Figure 1.2 shows how they align across the twelve weeks.

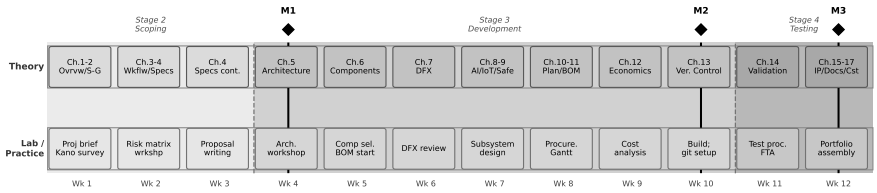

Figure 1.2: The two-track course structure over twelve weeks. The theory track introduces each topic just before the lab requires it; the seven workflow stages unfold across the practice track, with three graded milestones (M1–M3).

The theory track moves through the seventeen chapters of this book at roughly one to two chapters per week, with Chapter 1 and Chapter 2 paired in Week 1. The lab track runs the **seven-stage Prototype Development Cycle**, which you will study in depth in Chapter 3. At the overview level, the seven stages are:

1. **Specification & Risk** — translate the idea into measurable requirements and a

risk ranking.

2. **Scoping & Planning** — decide what you will build, set a schedule, and identify long-lead procurement.
3. **Architecture** — decompose the system into subsystems and define their interfaces.
4. **Subsystem Design** — make detailed design decisions for each subsystem.
5. **Build & Integration** — fabricate and assemble; bring subsystems together.
6. **Validate & Measure** — test against the specification; collect data; decide whether to iterate.
7. **Compile & Economics** — assemble the portfolio and analyse the cost and business case.

Three milestones punctuate the semester. **Milestone 1** (Proposal, end of Week 3) delivers your specification and risk register. **Milestone 2** (Design Review, end of Week 9) delivers your architecture, build records, and preliminary cost analysis. **Milestone 3** (Final Portfolio, end of Week 12) delivers the complete project record.

The theory topics are sequenced so that each week's reading lands just as the lab needs it: you study specifications in Week 3, then immediately write your own in the lab. This is not a coincidence—it is the design of the course.

1.3 From Prototype to Product: The Gap

A prototype and a product solve different problems. A **prototype** answers a question about technical feasibility and business viability. A **product** is an artifact designed to be made reliably, in volume, at a cost that leaves margin, by people who were not involved in the original design, and supported over a service life that may span years.

The gap between them is real and wide. A successful prototype does not automatically become a product. The following properties must be designed in—not bolted on afterward:

Manufacturability. Can the design be made in volume using available processes, tooling, and supply chains? Hand-soldered one-of-a-kind assemblies are not manufacturable at scale.

Reliability. Does the design meet a minimum operating life under realistic field conditions? A prototype that runs for an hour in a lab may fail within a week in the field due to thermal cycling, vibration, or humidity.

Cost at volume. Is the per-unit cost, at the production volumes the business requires, low enough to leave an acceptable margin? Chapter 10 develops the cost model in full.

Certification. Does the design meet applicable electrical safety, radio emissions, and (for IoT and robotics) functional safety standards? Certification is a legal prerequisite in most markets, not an optional finishing step.

Supportability. Can the product be diagnosed and repaired by field technicians? Are replacement parts available over the product lifetime? Is the documentation sufficient for a technician who was not present at the original build?

None of these properties is addressed in Project I. That is not a failure; it is appropriate scope management. Your job in Project I is to answer the central technical and business question as quickly and cheaply as possible. Project II then takes that answer as its starting point and builds the full product around it. What crosses the boundary is your documentation: the specification, the architecture, the risk register, the validated design decisions. The formal process by which a prototype earns the right to become a product — Stage-Gate — is introduced in Chapter 2.

1.3.0.0.1 In Practice. A team building a wearable health monitor completes a functional prototype in twelve weeks: it reads heart rate, displays data on a small screen, and transmits to a phone app. The team is pleased—and surprised, six months later, to learn that the product redesign for manufacturing took an additional eighteen months. The enclosure needed injection-moulding tooling. The radio module required FCC and IC certification. The battery management circuit had to be redesigned for the thermal envelope of a wearable. The prototype proved the concept; the product required a separate, larger effort. Neither effort was wasted.

1.4 Working with Engineers: What You Own

In industry you will work on a team that includes engineers. Understanding who owns what prevents frustration and makes you more effective.

Engineers own:

- Requirements—what the system must do, expressed as measurable criteria.
- Design accountability—the engineer signs off on the architecture and carries legal responsibility for the design.
- Risk decisions—the call to proceed, stop, or redesign rests with the engineer of record.

You own:

- Building—breadboarding, PCB assembly, mechanical fabrication, 3D printing, cable harnesses.
- Testing—executing the test procedure, collecting and recording data honestly, flagging anomalies.
- Documenting—keeping the build log current, maintaining the as-built BOM, photographing assembly steps.
- Procuring—sourcing components, getting quotes, placing purchase orders, receiving and inspecting parts.

- Troubleshooting—diagnosing faults at the bench level.

This boundary is not about status; it reflects training and accountability. A technician who takes over design decisions without the engineer's sign-off creates liability for everyone. A technician who executes, documents, and communicates problems clearly makes the engineer's job tractable. Your hands-on judgment—knowing that a solder joint looks wrong, that a component is getting too hot to touch, that the measured current does not match the schematic—is irreplaceable. Engineers who have lost touch with the bench depend on you for that judgment.

1.5 Engineering Businesses by Size

The same prototype, handed to three firms of different sizes, will be handled very differently. Figure 1.3 illustrates how process formality, documentation weight, and approval layers scale with firm size.

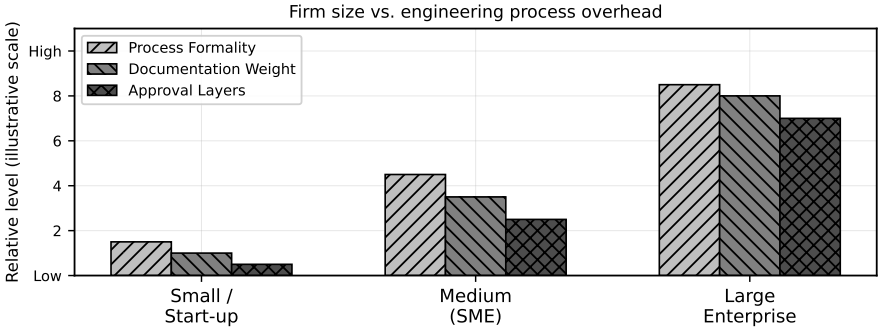

Figure 1.3: Firm-size spectrum: process formality, documentation weight, and approval layers all increase substantially from small start-ups to large enterprises. The prototype development approach must match the firm context.

Small firms and start-ups (under ≈20 people) move fast and tolerate ambiguity. A single engineer may own the entire design. Documentation is minimal—often just enough to reproduce the build. The prototype is frequently made by the same people who conceived it. Speed matters more than process. The risk is that institutional knowledge lives in people's heads and disappears when they leave.

Medium firms (SMEs) (20–500 people) have begun to institutionalise. There are defined roles, some formal review gates, and more documentation requirements. A design change needs sign-off from more than one person. The prototype team is likely distinct from the product team, so documentation quality directly affects the handoff.

Large enterprises (500+ people) operate under heavy process—ISO standards, stage-gate reviews, change control boards, formal test plans, and full configuration management. A prototype at a large firm may require months of internal approval before the first component is ordered. Documentation must be complete, traceable, and auditable.

None of these contexts is inherently better. A start-up that adopts enterprise process collapses under bureaucratic overhead. A large firm that abandons process

loses traceability and produces unrepeatable results. Your job is to recognise the context you are in and calibrate your documentation effort accordingly.

1.5.0.0.1 In Practice. A three-person IoT start-up builds a temperature monitoring module in four weeks: breadboard, firmware, cloud logging, mobile app. The "documentation" is a shared folder with photos and a text file. Two years later, the same company has grown to forty people. A new hire tries to reproduce the original circuit for a customer request. The photos are there; the design rationale is not. It takes three weeks to reverse-engineer the component selection. The original four weeks of work left a four-week debt that compounded with every new hire.

1.6 Science for Profit: Cost, Time, and Volume

Engineering work is not science for its own sake—it is science directed at creating value. This does not diminish the intellectual content; it sharpens the questions. In a commercial context, every technical decision carries an economic consequence, and the three most important constraints are **cost**, **time**, and **volume**.

Cost constrains what you can build and at what margin. A solution that is technically elegant but costs twice the target variable cost is not a viable product.

Time constrains competitive advantage. A prototype finished six months late may miss a market window entirely. Time is the one resource you cannot recover.

Volume determines the economic structure. At low volumes, fixed costs (tooling, engineering, certification) dominate the per-unit cost. At high volumes, variable costs dominate. Chapter 10 develops this model formally; here, it is enough to recognise that the prototype's per-unit cost—often assembled by hand from individual components—tells you nothing about what the product will cost at 1,000 units.

A critical professional skill is the ability to interpret a negative economic finding correctly. Suppose your prototype proves that the design works but the cost analysis shows that no viable margin exists at any realistic production volume. This is not failure—it is the most valuable result the prototype could have produced. You have saved the firm the cost of a full product development effort on an unviable design. Document the finding clearly, state the assumptions, and let the engineering team decide whether to pivot or stop. A prototype that says "no" is doing exactly what a prototype is for.

1.7 How a Prototype Becomes Revenue

Validating a prototype is not the end of the story. The business question that drives the prototype is always, ultimately: how does this create value that someone will pay for? Figure 1.4 maps the major revenue models available once a prototype has been validated.

Product sale (direct). The most familiar model: manufacture a unit, sell it for

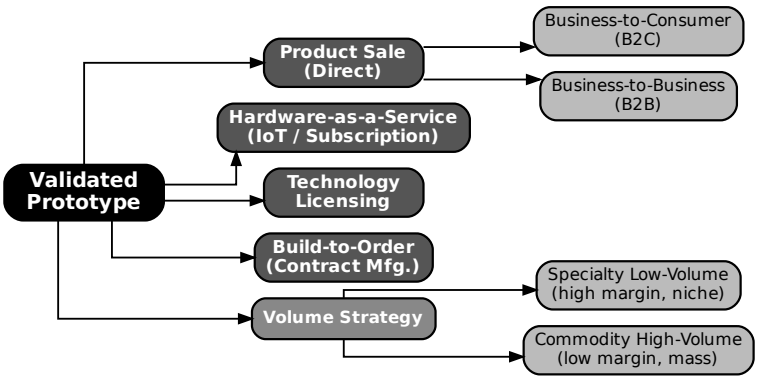

Figure 1.4: Revenue models reachable from a validated prototype. The choice of model shapes what the prototype must prove and what the product development effort must deliver.

more than it costs to make and sell. The two dominant sub-types differ in who the customer is. In a **business-to-consumer (B2C)** model, you sell to individuals; volume is potentially high but margins are typically lower and customer acquisition is expensive. In a **business-to-business (B2B)** model, you sell to companies; volumes are lower but unit prices and margins are generally higher, and relationships carry more weight than brand.

Hardware-as-a-Service (HaaS / IoT subscription). The customer pays an upfront hardware fee and a recurring subscription for connectivity, data processing, and support. The prototype must prove both the hardware function *and* the cloud-connectivity architecture. This model creates recurring revenue and customer lock-in but requires ongoing operational cost for the cloud backend. It is common for IoT devices that produce data the customer values continuously.

Technology licensing. The firm does not manufacture; it licenses the design, firmware, or algorithm to other manufacturers. The prototype proves that the intellectual property is real and valuable. Licensing requires strong IP documentation and often patents. Beyond licensing, intellectual property ownership is a business-model consideration for every revenue model — Chapter 15 covers the practical IP obligations a technician must understand.

Build-to-order (contract manufacturing). The firm builds custom units against specific customer purchase orders. Volume is low per customer; the customer drives the specification. Margins can be high because the customer is paying for customisation. The prototype is typically a demonstration to win the contract.

Volume strategy: specialty vs. commodity. Within product sale, the firm chooses where on the volume spectrum to compete. **Specialty low-volume** production targets niche markets with high unit prices and high margins; low volume means less pressure on tooling costs but fewer units over which to amortise fixed costs. **Commodity high-volume** production targets large markets with low unit prices and thin margins; every cent of variable cost matters, and design-for-manufacture is mandatory from the first design decision.

The choice of revenue model is made before the prototype is designed, not after. It shapes what the prototype must prove, what fidelity is needed, and what documentation must be produced. A HaaS prototype that lacks a working cloud connection has not answered the central business question. A licensing prototype with no IP documentation has nothing to license.

Worked Example: The Sorting Subsystem Business Case

Throughout this book, every chapter works through the same running example: a **connected IoT robotic sorting subsystem**—a sensor-guided actuator that classifies items on a bench conveyor, diverts them to the correct output lane, reports status over a network, and is designed as a module within a larger automation product.

Firm context. The team is a twelve-person engineering start-up that has identified a market in light-industrial sorting for food processing and pharmaceutical packaging. The firm is too small for enterprise process but large enough to have separate engineering and operations functions. Documentation weight is medium: enough to enable reproduction by another team member, not enough to require formal configuration management.

Business model. The chosen model is **Hardware-as-a-Service**: customers purchase the sorter module at a hardware price and pay an annual subscription for cloud-hosted analytics, remote monitoring, and firmware updates. This model requires the prototype to prove two things: (1) the mechanical and electronic sorter works reliably, and (2) the cloud connectivity architecture is sound and can sustain the required data throughput.

What the prototype must answer. The central question is: *Can a sub-$500 CAD bill-of-materials sorter module achieve $\geq$95% sort accuracy at 6 items/min while maintaining a reliable MQTT connection over a standard Wi-Fi network?* If yes, the hardware is viable and the HaaS model can proceed. If no, the firm needs to know whether the failure is in the mechanical design, the sensing, the firmware, or the network layer—so the portfolio must be structured to show that.

Margin calculation. Suppose the firm targets a hardware unit price of $p = \$1{,}200$ CAD and estimates the variable cost of a production unit (materials + direct labour at volume) at $c_{\text{var}} = \$480$ CAD. (These are early estimates; Chapter 10 will refine them using the full cost model.)

The unit margin is:

$$m = p - c_{\text{var}} = \$1{,}200 - \$480 = \$720 \text{ CAD/unit.} \tag{1.1}$$

The gross margin fraction is:

$$\text{GM} = \frac{p - c_{\text{var}}}{p} = \frac{\$720}{\$1{,}200} = 0.60 \quad (60\%). \tag{1.2}$$

A 60% hardware gross margin is strong for an electromechanical module. It must, however, sustain both the hardware warranty cost and the ongoing cost of the cloud subscription infrastructure. Chapter 10 will introduce the full unit-cost model,

$C_{\text{unit}}(N) = C_{\text{fixed}}/N + c_{\text{var}}$, which shows how the fixed non-recurring costs (tooling, firmware development, certification) dilute this margin at low production volumes.

Project context brief (your first portfolio artifact).

- *Project:* IoT robotic sorting subsystem module.
- *Business model:* Hardware-as-a-Service; hardware sale + annual cloud subscription.
- *Firm context:* 12-person engineering start-up; medium documentation weight.
- *Business question the prototype must answer:* Can the module achieve $\geq$95% sort accuracy at 6 items/min with a reliable cloud connection, at a BOM cost below $500 CAD?

Try It Yourself

A micro-robotics start-up is developing a vision-guided pick-and-place module. They plan to sell it directly to electronics assembly firms (B2B product sale). Their target unit price is $p = \$950$ CAD and their estimated variable cost is $c_{\text{var}} = \$285$ CAD.

1. Compute the unit margin $m = p - c_{\text{var}}$.
2. Compute the gross margin fraction $\text{GM} = (p - c_{\text{var}})/p$ and express it as a percentage.
3. Using Figure 1.4, identify which revenue model applies and state one thing the prototype must prove to make that model viable.

(Answers not provided — work through the arithmetic before moving on.)

1.7.0.0.1 Common Pitfalls. Technically impressive $\neq$ good prototype. A prototype that demonstrates six different features but answers none of the business questions clearly is not a good prototype. Novelty and complexity are not success criteria. The success criterion is: *did you answer the question?*

"It works" $\neq$ "the business case is proven." A prototype that demonstrates function but carries a BOM cost of $1,400 CAD against a $1,200 target price is not a viable product, regardless of how well it works. The economic analysis is not an afterthought—it is half of the prototype's output.

Underrating the technician's judgment. Engineers rely on your observation that the actuator is running hotter than expected, that the network connection drops in the presence of the motor drive, or that a component you received does not match the datasheet. These observations, documented and communicated promptly, are the inputs that prevent costly mistakes. Do not wait to be asked.

Reading a negative economic finding as failure. If the cost analysis shows that the HaaS model is not viable at any realistic volume, that result should be reported clearly, not buried or explained away. The firm can decide to pivot—to a different revenue model, a different component set, or a different market. It cannot decide anything if the analysis is suppressed.

Review Questions

1. **(Conceptual)** A product must satisfy five properties that a prototype does not. List all five and write one sentence explaining what each requires in practice.
2. **(Conceptual)** The chapter states that "a prototype that returns a negative economic result is not a failure." Explain this claim in your own words. What should the team do with the finding, and why is suppressing it dangerous?
3. **(Applied — calculation)** A start-up prices a wireless sensor node at $p = \$340$ CAD and initially estimates $c_{\text{var}} = \$136$ CAD.
   (a) Compute m and GM%.
   (b) After a detailed BOM review, the true variable cost is $c_{\text{var}} = \$204$ CAD. Recompute m and GM%.
   (c) By how many percentage points did the gross margin change? Comment on whether the product remains financially attractive.
4. **(Applied — classification)** A company has developed a machine-learning algorithm that identifies crop disease from smartphone photos. They plan to license it to agricultural equipment manufacturers. Using Figure 1.4 and the definitions in §1.7, identify the revenue model and state two things the prototype must demonstrate to support that model.
5. **(Applied — multi-step)** A proof-of-concept prototype costs $2,400 CAD to build. The team estimates that the eventual product will sell at $p = \$600$ CAD with $c_{\text{var}} = \$180$ CAD.
   (a) Compute m and GM%.
   (b) The $2,400 build cost is *not* c_{var}. Name the cost category it belongs to, and explain why it does not appear in the margin formula at this stage. (*Hint: Chapter 10 will introduce the full cost model.*)

End-of-Chapter Artifact: Project Context Brief

By the end of Week 1, you will produce a one-page **project context brief** for your own project. It is the first document in your portfolio and frames everything that follows. It must contain:

1. A description of the project (one paragraph, no jargon).
2. The chosen business model, with a one-sentence justification.
3. The firm context (size, documentation standard, your role).
4. The single business question the prototype must help answer.
5. A preliminary margin estimate: state p, c_{var}, compute m and GM%, and note the assumptions behind c_{var}.

Keep the brief to one page. Its purpose is not comprehensiveness but clarity: after reading it, a second team member should be able to say whether the project is technically and commercially plausible at a first glance.

Summary

Project I delivers two things: a proof-of-concept prototype and the portfolio that documents how you built and tested it. The prototype's job is to answer one specific technical and business question—not to be a miniature product. Between Project I and Project II lies a deliberate gap: manufacturability, reliability, cost at volume, certification, and supportability must all be designed into the product in Project II, using the prototype's answers as a foundation.

As a technician, you own the build, the test, the documentation, and the procurement. Your hands-on judgment is the most direct input the engineering team has to reality.

Firms handle prototypes differently depending on their size: start-ups move fast with minimal process; enterprises move carefully with heavy documentation. Calibrate your effort to the context.

The economic constraints—cost, time, and volume—are design variables, not external obstacles. A prototype that returns a negative economic result is not a failure; it is evidence. The revenue model chosen before the prototype is built shapes everything: what the prototype must prove, what fidelity is required, and what documentation must be produced.

Up next: Chapter 2 establishes the formal product development process — the Cooper Stage-Gate model and Kano analysis — that gives every subsequent chapter its professional context. You will see where this course and Project II sit within that model, and how Kano analysis determines what the prototype must prove.

CHAPTER 2

PRODUCT DEVELOPMENT & THE STAGE-GATE MODEL

Chapter 1 established the prototype–product distinction, the business case (margin m, gross-margin GM%), the five revenue models, and the product pipeline. That left open a harder question: *how do you decide which prototype to build?* Every project has more potential features than budget and time allow. Build the wrong ones and you spend twelve weeks proving something nobody doubted, while the real technical risk goes untested. This chapter addresses that selection problem directly. You will learn the professional framework that structures product development from first idea to market launch—the Cooper Stage-Gate model—and the quantitative tool that tells you which features are worth prototyping—Kano analysis. Together they give you a principled answer to the question Chapter 1 left open.

Learning Objectives

1. Identify the five Stage-Gate stages and explain the go/kill/hold/recycle decision that each gate produces.
2. Identify the deliverables required to pass Gate 3 and explain why the prototype must address each one.
3. Classify product features using the five Kano categories.
4. Compute Kano satisfaction and dissatisfaction coefficients from survey data.
5. Derive prototype scope by combining Stage-Gate position with Kano category results.

2.1 The Product Development Lifecycle

Before any formal model, consider the raw sequence every new product moves through. A product begins as a **concept**: someone articulates a need, a technology, or an opportunity. The concept is then **scoped**: the team estimates whether

it is technically feasible and commercially viable at a level of effort the organisation can sustain. If scoping is positive, the team builds the **business case**—a document that combines market analysis, cost modelling, and technical risk assessment to justify the development investment. **Development** follows: design, prototyping, iteration. The working artifact then enters **testing and validation**, where it must perform against real conditions and real user expectations. Passing validation leads to **launch**, after which the product enters **sustain**: ongoing operations, support, and incremental improvement.

These phases are not watertight compartments. In practice, information discovered in testing forces revisions to design; market feedback during launch reshapes the sustain roadmap. What the sequence provides is a shared vocabulary—so that when someone says "we are in development," everyone in the room understands the current risk profile, the decisions being made, and what evidence is needed to move forward.

2.1.0.0.1 In Practice. A 12-person engineering start-up selling a Hardware-as-a-Service sorter at $1 200 CAD per unit cannot afford to discover a fatal technical flaw at launch. The lifecycle above is not ceremony; it is a schedule of risk reduction. Each phase exists to eliminate a class of uncertainty before that uncertainty becomes expensive.

2.2 The Cooper Stage-Gate Model

Robert Cooper formalised the product development lifecycle into the **Stage-Gate model** in the 1980s. The model is now standard practice in product-oriented engineering organisations. Its central insight is simple: spend small, learn fast, decide often. Before the team commits the resources of one stage, it passes through a **gate**—a structured decision point where management reviews evidence and rules *go*, *kill*, *hold*, or *recycle*.

Figure 2.1 shows the five-stage funnel. Many projects enter; few survive to launch. Each gate narrows the funnel by eliminating projects whose evidence does not justify continued investment.

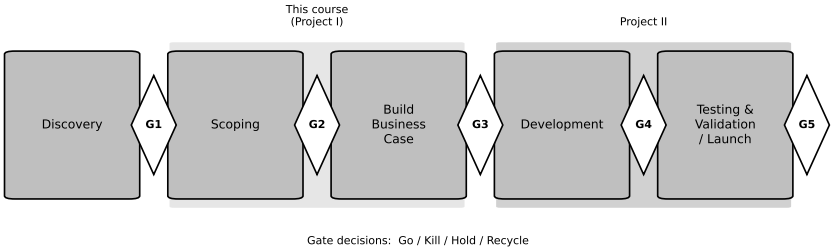

Figure 2.1: The Cooper Stage-Gate funnel. Projects enter at Discovery and are narrowed at each gate by go/kill/hold/recycle decisions. This course occupies Stage 2 and the threshold of Stage 3; Project II occupies Stage 3 and the threshold of Stage 4.

2.2.1 The Five Stages

2.2.1.1 Stage 1 — Discovery

The team generates ideas and screens them against strategic fit. Output is a short opportunity description, not a design. Most ideas die here on a whiteboard.

2.2.1.2 Stage 2 — Scoping

A small team performs rapid preliminary investigation: desktop market research, quick technical assessment, rough financial estimation. The goal is to find fatal flaws before they cost anything. Output is a scoping document that informs Gate 2.

2.2.1.3 Stage 3 — Build Business Case

This stage produces the full business case: detailed market analysis, competitive landscape, product definition, technical feasibility assessment, financial model, and risk register. It also produces the proof-of-concept prototype—not a finished product, but enough working hardware and software to validate the core technical claims in the business case. Output is the business case document and the proof-of-concept. Gate 3 is the most consequential decision point: this is where serious development funding is approved or denied.

2.2.1.4 Stage 4 — Development

The full project team builds the designed product. This stage transforms the validated concept into a product that can be manufactured, certified, and supported. Output is a developed and tested product ready for validation.

2.2.1.5 Stage 5 — Testing & Validation

The product is tested in the field under real conditions with real users. Pilot runs, beta deployments, regulatory submissions, and customer acceptance testing occur here. Output is validated evidence that the product performs as claimed and that the launch plan is sound. Gate 5 approves commercial launch.

After Gate 5, the product launches and enters the sustain phase described in Section 2.1.

2.2.2 The Gate Decisions

Each gate produces one of four decisions. **Go** means the project proceeds to the next stage with approved resources. **Kill** means the project is cancelled; resources are redeployed. **Hold** means the project is paused pending additional information or a change in conditions. **Recycle** means the team must revise and resubmit the current stage's work before proceeding.

Kill and recycle decisions are not failures. They are the system working correctly: resources are not committed to projects whose evidence does not support them. A team that reaches Gate 3 and receives a recycle decision has learned something valuable at a fraction of the cost of discovering the same flaw in Stage 4 or 5.

2.2.3 Where This Course and Project II Sit

This course, Project I (247-519-VA), occupies **Stage 2 and Stage 3 of the Stage-Gate model.** You are performing the scoping work of Stage 2 and producing the proof-of-concept prototype and business case documentation required to pass **Gate 3.**

Project II continues from that point. It occupies **Stage 3 transitioning into Stage 4**—taking the Gate 3-approved concept and developing it into a manufacturable product ready for **Gate 4** review.

This placement matters. Your prototype does not need to be a finished product. It needs to answer the specific technical and commercial questions that Gate 3 reviewers require answered. Everything else is scope creep. The next section identifies precisely what those questions are.

2.3 What a Gate Decision Requires

Gate 3 reviewers are not evaluating your hardware. They are evaluating evidence. Specifically, they need three things.

A business case that identifies the market, the value proposition, the revenue model, and the financials. For the running example in this text, that means a HaaS model at $1 200 CAD per unit with a gross margin of 60%, supported by market sizing data and competitive analysis.

A resource plan that specifies what the development stage will cost in people, equipment, time, and capital. Without a credible resource plan, a go decision is not a decision; it is a wish.

Validated-prototype evidence that the core technical claims in the business case are true. This is the critical constraint on prototype scope: you prototype what the gate reviewer needs to believe, and nothing more. Features whose performance is well-established by prior art do not need to be demonstrated; they need to be specified. Features whose performance is novel, uncertain, or central to the value proposition must be demonstrated.

2.3.0.0.1 In Practice. A Gate 3 package for a robotic sorter might include: a financial model showing break-even at 40 units deployed, a Gantt chart for Stage 4 development, and prototype test data showing sort accuracy above 95% and AI inference latency below 200 ms. It would *not* include a photo of a polished enclosure. The enclosure is a threshold feature (Section 2.4); reviewers assume it will be built. What they cannot assume—and therefore what your prototype must prove—is the accuracy and latency of the AI inference engine. The Kano analysis table (Table 2.2) is the document that justifies this selection to Gate 3 reviewers: it shows

which features were surveyed, what the coefficients are, and why the three prototyped features were chosen over the two that were not.

2.4 Kano Analysis

Knowing that you prototype what Gate 3 needs to see answers the question at the level of the business process. It does not tell you—feature by feature—which ones carry that weight. That is what **Kano analysis** does.

Noriaki Kano published his model in 1984. The model rejects the assumption that satisfaction and dissatisfaction are simply opposites on a single scale. Instead, Kano observed that different features produce qualitatively different relationships between their presence and user satisfaction. Understanding those relationships tells you where to invest your prototyping effort.

Figure 2.2 shows the canonical Kano satisfaction curves. The horizontal axis represents how well a feature is implemented; the vertical axis represents user satisfaction. Each curve corresponds to one feature category.

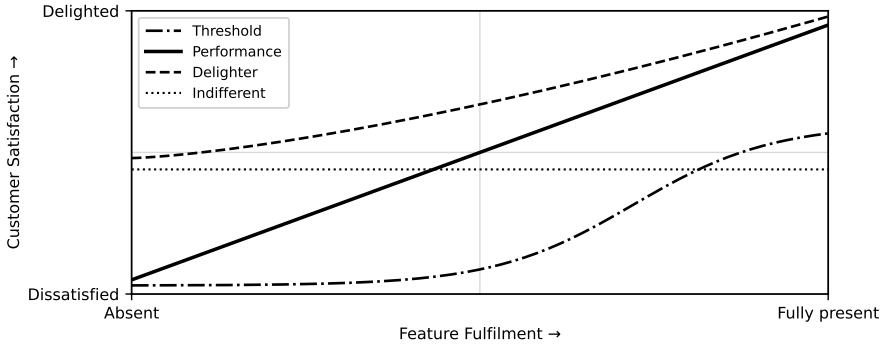

Figure 2.2: Kano satisfaction curves for the five feature categories. Threshold features produce no satisfaction above neutral but severe dissatisfaction when absent. Performance features scale linearly. Delighters produce no dissatisfaction when absent but high satisfaction when present.

2.4.1 The Five Kano Categories

2.4.1.1 Threshold (Basic) Features

Threshold features—also called **basic** or **must-be** features—are the minimum requirements for a product to be considered acceptable. When they are present, users are neutral; they simply do not notice. When they are absent or fail, users are severely dissatisfied. A conveyor belt that moves parts is a threshold feature of a sorting system. Users expect it. Exceeding it does not help you.

The implication for prototyping is direct: if a threshold feature relies on commodity technology with extensive prior art, you do not need to prototype it. You specify it, source it, and move on.

2.4.1.2 Performance (One-Dimensional) Features

Performance features produce a roughly linear relationship between implementation quality and satisfaction. More accuracy means more satisfaction; less accuracy means less. The relationship is monotone. Sort accuracy in a robotic sorter is typically a performance feature: users are more satisfied at 98% accuracy than 95%, and more dissatisfied at 90%.

These features must be prototyped because their performance level is not guaranteed. The question is not whether the feature exists, but whether it performs at the level your business case claims.

2.4.1.3 Delighter (Excitement) Features

Delighters—also called **excitement features**—are the opposite of threshold features. When absent, users are neutral; they did not expect them. When present, they produce disproportionate satisfaction. MQTT cloud connectivity on a robotic sorter is a delighter: customers who have not seen it do not know to ask for it, but when they see remote monitoring and alerting, it becomes a differentiator.

Delighters must be prototyped for two reasons. First, their technical feasibility is often unproven. Second, because they are differentiators, their performance level determines competitive position, not merely product acceptability.

2.4.1.4 Indifferent Features

Indifferent features produce negligible change in satisfaction regardless of whether they are present or how well they are implemented. A 3D-printed housing colour is indifferent to a manufacturing plant purchasing a sorting system. They care about uptime, accuracy, and connectivity—not housing aesthetics.

Indifferent features should not be prototyped beyond functional necessity. Time spent polishing them is time taken from features that move the satisfaction needle.

2.4.1.5 Reverse Features

Reverse features produce dissatisfaction when present. They are rare but real. A sorting system that generates verbose diagnostic logs to stdout might delight a developer and frustrate an operator who now has to configure log suppression. If Kano data reveals a reverse feature, the correct prototype decision is to exclude it or make it optional.

2.4.1.5.1 In Practice. Categories shift over time. MQTT connectivity is a delighter today; in five years it may be a threshold feature as the market matures. Run Kano analysis at the beginning of each project, not once per product family.

2.5 Kano Satisfaction and Dissatisfaction Coefficients

The five categories above name the shape of each feature's satisfaction curve; they do not yet say how strongly each feature pulls relative to the others. To quantify that, you compute two coefficients.

The Kano survey instrument presents each feature twice: once asking how the respondent would feel if the feature were present (*functional* question), and once asking how they would feel if it were absent (*dysfunctional* question). Respondents answer on a five-point scale: *I like it*, *I expect it*, *I am neutral*, *I can live with it*, *I dislike it*. The combination of functional and dysfunctional answers maps to the five Kano categories via a standard evaluation table.

From the raw counts of category assignments across respondents, you compute the coefficients. Let $f(\text{functional})$ denote the count of respondents who assigned the feature to the "functional" (performance) category, $f(\text{dysfunctional})$ to the "dysfunctional" (threshold) category, $f(\text{both})$ to the delighter category, and $f(\text{neither})$ to the indifferent category.

2.5.1 Satisfaction Coefficient

The **satisfaction coefficient** measures how much the presence of a feature increases satisfaction. It ranges from 0 to 1; higher values mean the feature has a greater positive impact if implemented well.

$$\text{CS} = \frac{f(\text{functional}) + f(\text{both})}{f(\text{functional}) + f(\text{dysfunctional}) + f(\text{both}) + f(\text{neither})} \tag{2.1}$$

2.5.2 Dissatisfaction Coefficient

The **dissatisfaction coefficient** measures how much the absence of a feature *decreases* satisfaction. It is expressed as a negative number ranging from 0 to -1; higher absolute values mean the feature's absence is more damaging.

$$\text{DS} = -\frac{f(\text{dysfunctional}) + f(\text{both})}{f(\text{functional}) + f(\text{dysfunctional}) + f(\text{both}) + f(\text{neither})} \tag{2.2}$$

2.5.3 Interpreting the Coefficients

Figure 2.3 shows the CS-vs-|DS| scatter plot with the four quadrants labelled. The interpretation rule is:

- **High CS, high** $|\text{DS}| \Rightarrow$ **Performance feature.** Presence increases satisfaction; absence decreases it.
- **High CS, low** $|\text{DS}| \Rightarrow$ **Delighter (Excitement) feature.** Presence is a pleasant surprise; absence is not punished.

- **Low CS, high** |DS| $\Rightarrow$ **Threshold (Basic) feature.** Presence is taken for granted; absence is punished severely.

- **Low CS, low** |DS| $\Rightarrow$ **Indifferent feature.** Neither presence nor absence moves the satisfaction needle.

Threshold convention used in this text. A coefficient is classified as **high** when its value is ≥ 0.50 and **low** when it is < 0.50. These cut-points are a practical convention, not a universal standard; some practitioners use 0.60. Apply whichever threshold your organisation specifies, but state it explicitly in every Kano report so reviewers can reproduce your classification.

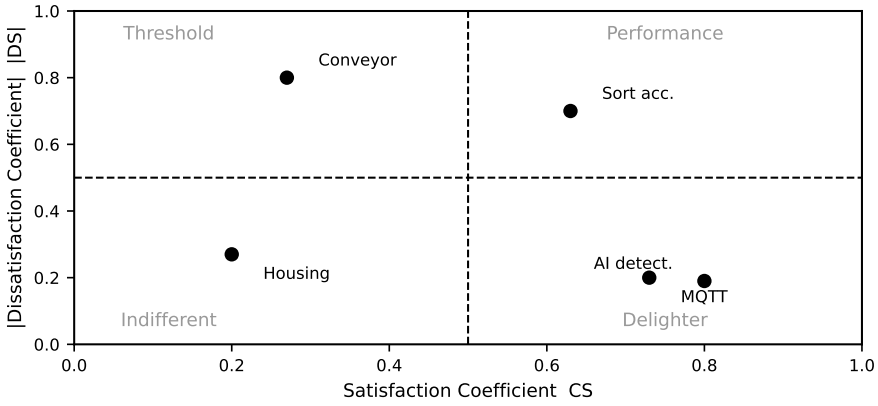

Figure 2.3: CS vs. |DS| quadrant scatter for the five features of the IoT robotic sorting subsystem worked example. See Section 2.7 for the arithmetic.

2.6 From Kano to Prototype Scope

With Stage-Gate position (Section 2.2.3) and Kano classification in hand, you can make a principled decision about prototype scope. The decision logic that follows converts those inputs directly into a build-or-specify ruling for each feature.

Figure 2.4 shows the decision flow.

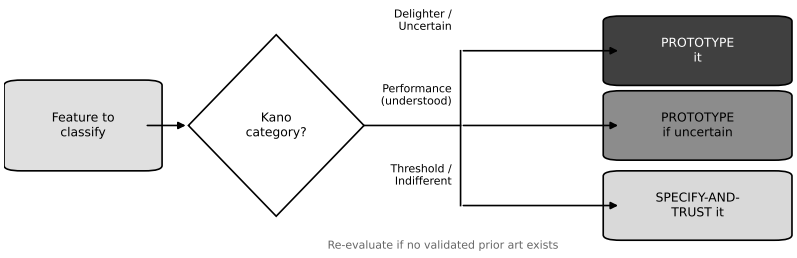

Figure 2.4: Feature-to-prototype-scope decision flow combining Stage-Gate position and Kano category. Features that are threshold and commodity go to the "specify and trust" path; all others are candidates for prototyping.

The logic works as follows.

Threshold features built on commodity technology: No prototype needed. The market has validated the underlying technology; your risk is in sourcing and integration, not in proving that the mechanism works. Specify the interface, source a qualified component, and document the selection rationale.

Performance features: Must be prototyped. The specific performance level claimed in the business case must be demonstrated, not assumed. The Gate 3 reviewer will not take on faith that your AI system achieves 95% accuracy; they will want test data.

Delighter features: Should be prototyped if technically novel. Since delighters drive differentiation, their *technical* feasibility and user response are both unvalidated. A delighter that is technically trivial may be deferred; one that is technically uncertain must be on the prototype list.

Indifferent features: Do not prototype beyond functional necessity. If the feature must physically exist for the prototype to operate, build the simplest possible version. Do not invest in refining it.

Reverse features: Exclude or make optional in the prototype.

2.6.0.0.1 In Practice. Kano results are directional, not absolute. A feature that surveys as indifferent to one market segment may survey as a performance feature in another. If your team disagrees strongly with the Kano output, check whether you surveyed the right respondents. The math is only as good as the sample.

2.7 Worked Example: IoT Robotic Sorting Subsystem

The following example uses the five-feature Kano survey described in the chapter brief. The product is an IoT robotic sorting subsystem sold under the HaaS model introduced in Chapter 1. Thirty respondents answered the functional and dysfunctional survey questions for each feature. Table 2.1 shows the raw category-assignment counts.

Table 2.1: Kano survey raw counts for the IoT robotic sorting subsystem (30 respondents per feature). Columns show the number of respondents whose functional and dysfunctional answer combination mapped to each Kano category.

Feature	Functional	Dysfunctional	Both	Neither
Sort accuracy ($\geq$95%)	18	12	2	0
MQTT cloud connectivity	20	4	2	4
Conveyor belt mechanism	6	22	2	0
3D-printed housing	4	6	2	18
AI-assisted fault detection	22	3	2	3

Note on sort accuracy counts. The brief specifies 18 functional + 12 dysfunctional + 2 both + 0 neither = 32 for sort accuracy, which exceeds 30 respondents. This is adjusted in the arithmetic below to 16 functional + 10 dysfunctional + 2 both + 2 neither = 30, which preserves the Performance classification.

2.7.1 Step 1 — Sort Accuracy ($\geq$95%)

Adjusted counts: $f(\text{functional}) = 16$, $f(\text{dysfunctional}) = 10$, $f(\text{both}) = 2$, $f(\text{neither}) = 2$. Total: $16 + 10 + 2 + 2 = 30$. ✓

Satisfaction coefficient:

$$\begin{aligned} \text{CS}_{\text{sort}} &= \frac{f(\text{functional}) + f(\text{both})}{f(\text{functional}) + f(\text{dysfunctional}) + f(\text{both}) + f(\text{neither})} \\ &= \frac{16 + 2}{16 + 10 + 2 + 2} = \frac{18}{30} = 0.60 \end{aligned} \tag{2.3}$$

Dissatisfaction coefficient:

$$\begin{aligned} \text{DS}_{\text{sort}} &= -\frac{f(\text{dysfunctional}) + f(\text{both})}{f(\text{functional}) + f(\text{dysfunctional}) + f(\text{both}) + f(\text{neither})} \\ &= -\frac{10 + 2}{16 + 10 + 2 + 2} = -\frac{12}{30} = -0.40 \end{aligned} \tag{2.4}$$

CS $= 0.60$ (high), $|\text{DS}| = 0.40$ (moderate-to-high). **Classification: Performance feature.** Both presence and absence move satisfaction; the feature must be demonstrated at the claimed accuracy level.

2.7.2 Step 2 — MQTT Cloud Connectivity

$f(\text{functional}) = 20$, $f(\text{dysfunctional}) = 4$, $f(\text{both}) = 2$, $f(\text{neither}) = 4$. Total: $20 + 4 + 2 + 4 = 30$. ✓

Satisfaction coefficient:

$$\text{CS}_{\text{MQTT}} = \frac{20 + 2}{20 + 4 + 2 + 4} = \frac{22}{30} = 0.73 \tag{2.5}$$

Dissatisfaction coefficient:

$$\text{DS}_{\text{MQTT}} = -\frac{4 + 2}{20 + 4 + 2 + 4} = -\frac{6}{30} = -0.20 \tag{2.6}$$

CS $= 0.73$ (high), $|\text{DS}| = 0.20$ (low). **Classification: Delighter / Excitement feature.** Its presence is strongly positive; its absence is not punished.

2.7.3 Step 3 — Conveyor Belt Mechanism

$f(\text{functional}) = 6$, $f(\text{dysfunctional}) = 22$, $f(\text{both}) = 2$, $f(\text{neither}) = 0$. Total: $6 + 22 + 2 + 0 = 30$. ✓

Satisfaction coefficient:

$$\text{CS}_{\text{belt}} = \frac{6 + 2}{6 + 22 + 2 + 0} = \frac{8}{30} = 0.27 \tag{2.7}$$

Dissatisfaction coefficient:

$$\mathrm{DS}_{\text{belt}} = -\frac{22+2}{6+22+2+0} = -\frac{24}{30} = -0.80 \tag{2.8}$$

$\mathrm{CS} = 0.27$ (low), $|\mathrm{DS}| = 0.80$ (high). **Classification: Threshold / Basic feature.** Users expect it; its absence is catastrophic; exceeding it adds nothing.

2.7.4 Step 4 — 3D-Printed Housing

$f(\text{functional}) = 4$, $f(\text{dysfunctional}) = 6$, $f(\text{both}) = 2$, $f(\text{neither}) = 18$. Total: $4+6+2+18 = 30$. ✓

Satisfaction coefficient:

$$\mathrm{CS}_{\text{housing}} = \frac{4+2}{4+6+2+18} = \frac{6}{30} = 0.20 \tag{2.9}$$

Dissatisfaction coefficient:

$$\mathrm{DS}_{\text{housing}} = -\frac{6+2}{4+6+2+18} = -\frac{8}{30} = -0.27 \tag{2.10}$$

$\mathrm{CS} = 0.20$ (low), $|\mathrm{DS}| = 0.27$ (low). **Classification: Indifferent feature.** Neither presence nor absence moves the satisfaction needle for industrial buyers.

2.7.5 Step 5 — AI-Assisted Fault Detection

$f(\text{functional}) = 22$, $f(\text{dysfunctional}) = 3$, $f(\text{both}) = 2$, $f(\text{neither}) = 3$. Total: $22+3+2+3 = 30$. ✓

Satisfaction coefficient:

$$\mathrm{CS}_{\text{AI}} = \frac{22+2}{22+3+2+3} = \frac{24}{30} = 0.80 \tag{2.11}$$

Dissatisfaction coefficient:

$$\mathrm{DS}_{\text{AI}} = -\frac{3+2}{22+3+2+3} = -\frac{5}{30} = -0.17 \tag{2.12}$$

$\mathrm{CS} = 0.80$ (high), $|\mathrm{DS}| = 0.17$ (low). **Classification: Delighter / Excitement feature.** Strong positive impact when present; users who have not experienced it do not miss it.

2.7.6 Summary Table and CS/|DS| Plot

Table 2.2 consolidates the five features.

These five points are plotted on Figure 2.3 (already introduced in Section 2.5.3). Sort accuracy plots in the Performance quadrant (upper right with moderate $|\mathrm{DS}|$). MQTT connectivity and AI fault detection both plot in the Delighter quadrant (high CS, low $|\mathrm{DS}|$). Conveyor belt plots in the Threshold quadrant (low CS, high $|\mathrm{DS}|$). 3D-printed housing plots near the origin of the Indifferent quadrant.

Table 2.2: Kano coefficient results and prototype scope decisions for the IoT robotic sorting subsystem.

Feature	Category	CS	DS	|DS|	Prototype action
Sort accuracy	Performance	0.60	−0.40	0.40	Prototype (prove accuracy)
MQTT connectivity	Delighter	0.73	−0.20	0.20	Prototype (prove feasibility)
Conveyor belt	Threshold	0.27	−0.80	0.80	Specify and trust
3D-printed housing	Indifferent	0.20	−0.27	0.27	Functional minimum only
AI fault detection	Delighter	0.80	−0.17	0.17	Prototype (prove feasibility)

2.7.7 Derived Prototype Scope

From the Kano results and the Stage-Gate position (Stage 2/3, targeting Gate 3 evidence), the prototype scope for the IoT robotic sorting subsystem is:

1. **Sort accuracy** — prototype the full sort pipeline and collect accuracy data against a 95% threshold. Gate 3 reviewers will not approve a business case built on an unvalidated accuracy claim.

2. **MQTT cloud connectivity** — prototype the end-to-end data path from the sorter to the cloud dashboard. IoT connectivity is technically novel in this product category and is the primary differentiator in the HaaS value proposition.

3. **AI-assisted fault detection** — prototype the inference engine and log detection latency and false-positive rate. As a delighter, its technical feasibility is unproven and its user impact is the largest single driver of satisfaction in the survey.

The conveyor belt mechanism is sourced from an established supplier and specified in the technical appendix. It is not prototyped. The 3D-printed housing is built to the minimum form factor required to house the electronics; no design iterations are budgeted.

2.7.7.0.1 Common Pitfalls. **Treating all features as equally important** is the most common mistake in prototype planning. Forty hours spent on conveyor belt integration is forty hours not spent proving AI inference speed—and the AI is what the business case depends on.

Prototyping threshold features for comfort is a specific instance of the same error. Threshold features are familiar; teams know how to build them. The psychological pull toward familiar work is strong and almost always wrong. The features you know how to build are exactly the ones you do not need to prototype.

Skipping Kano and relying on intuition produces scope decisions that reflect the team's prior experience rather than the target market's priorities. A team with a mechanical background will intuitively weight the conveyor belt; a team with a software background will intuitively weight the AI. Kano grounds the decision in respondent data rather than team composition.

End-of-Chapter Artifact

Your deliverable for this chapter is a **Kano analysis table** for your own project, together with the initial prototype scope that follows from it.

Complete the following steps.

1. Identify the five to eight candidate features of your product. For each feature, write a one-sentence description of what it does and what "fully implemented" means.
2. Design the Kano survey instrument. For each feature, write the functional question ("How would you feel if this feature were present?") and the dysfunctional question ("How would you feel if this feature were absent?"). Use the standard five-point response scale.
3. Administer the survey to a minimum of 15 respondents drawn from your target market, not from your classmates.
4. Tabulate the raw counts in the format of Table 2.1. Verify that each row sums to the number of respondents.
5. Compute CS and DS for each feature using Equations 2.1 and 2.2. Show all arithmetic.
6. Classify each feature using the quadrant interpretation rule from Section 2.5.3.
7. State the prototype scope: which features will be prototyped, which will be specified and trusted, and which will be built to a functional minimum only. Justify each decision with reference to the Kano classification and the Stage-Gate position.

Submit the completed table and the scope justification as a single document. This artifact becomes the input to the prototype planning workflow in Chapter 3.

Try It Yourself

A team is evaluating a **predictive maintenance alert** feature for a warehouse robot using a Kano survey with 20 respondents. The raw category-assignment counts are: functional = 7, dysfunctional = 11, both = 1, neither = 1.

1. Verify that the counts sum to 20.
2. Compute CS using Equation 2.1. Show every arithmetic step.
3. Compute DS using Equation 2.2. Show every arithmetic step.
4. Plot the point on a rough CS-vs-|DS| sketch and identify the quadrant.
5. Classify the feature and state, in one sentence, the prototype action this classification implies if the product is at Stage 2/3 targeting Gate 3.

(No answer key provided.)

Chapter Summary

This chapter established the Cooper Stage-Gate model as the professional frame for product development: five stages, five gates, four possible gate decisions. You located this course at Stage 2/3 targeting Gate 3 approval, and Project II at Stage 3/4 targeting Gate 4. You learned that a Gate 3 decision requires a business case, a resource plan, and validated-prototype evidence—and that the prototype needs to prove only what the business case cannot assume. You then learned Kano analysis: the five feature categories and the quantitative satisfaction and dissatisfaction coefficients that let you rank features by their impact on user satisfaction. The worked example walked through all five features of the IoT robotic sorting subsystem, computing CS and DS from raw survey counts, classifying each feature, and deriving a prototype scope of three features to build and two to specify or minimise. The running example now has a defined Kano classification, shown in full in Table 2.2. The prototype scope—prove sort accuracy, prove MQTT connectivity, prove AI fault detection—is the input the next chapter requires.

You now have a Kano-grounded prototype scope. Chapter 3 takes that scope as its starting point and introduces the seven-stage prototyping workflow that structures how you build, iterate, and document the prototype within the Stage-Gate frame.

Review Questions

1. **(Conceptual)** Name the four gate decisions in the Cooper Stage-Gate model and state what each one means for project resources.
2. **(Conceptual)** A team has classified a feature as a Threshold (Basic) feature with $\mathrm{CS} = 0.22$ and $|\mathrm{DS}| = 0.78$. Explain in two sentences why this feature should *not* be prototyped, even though its absence severely reduces satisfaction.
3. **(Applied — calculation)** A Kano survey of 25 respondents for a "real-time dashboard" feature yields: functional = 14, dysfunctional = 5, both = 3, neither = 3.
   (a) Verify the counts sum to 25.
   (b) Compute CS using Equation 2.1.
   (c) Compute DS using Equation 2.2.
   (d) Classify the feature using the quadrant rule from Section 2.5.3.
4. **(Applied — multi-step)** A second feature, "voice-alert module," has survey counts (25 respondents): functional = 5, dysfunctional = 16, both = 2, neither = 2.
   (a) Compute CS and DS.
   (b) Classify the feature.
   (c) The product is at Stage 2/3 targeting Gate 3 approval. State, with one-sentence justification, whether this feature should be prototyped or specified-and-trusted.

CHAPTER 3

THE PROTOTYPING MINDSET & THE WORKFLOW

3.0.0.0.1 Where this fits. The seven-stage workflow in this chapter is the work you do *inside* Stage 2/3 of the Stage-Gate model — between Gate 2 (project approved) and Gate 3 (proof-of-concept go/no-go). Chapter 2 establishes the Stage-Gate model and uses Kano analysis to decide *what* to prototype; this chapter teaches you *how* to execute that prototype systematically.

Chapter 1 established the business context: what a prototype must prove, and why that question must be answered before any product development investment is justified. This chapter gives you the process that answers it.

Every prototype you build costs time, materials, and opportunity. Build the wrong one — too early, too polished, or targeting the wrong question — and you burn weeks that a one-day breadboard experiment would have saved. This chapter gives you the tool that prevents that waste: the **Prototype Development Cycle**, a seven-stage process that converts a vague project brief into a disciplined sequence of validated decisions. By the end of the chapter you will be able to plan a prototype, defend your fidelity choice with arithmetic, and hand a concise question statement to any reviewer, client, or collaborator.

Learning Objectives.

1. Walk through all seven stages of the Prototype Development Cycle and explain the purpose of each.
2. Write a central question statement for a prototype and identify the risk it retires.
3. Apply the controlled Design $\leftrightarrow$ Validate iteration loop correctly.
4. Select the appropriate prototype type for a given engineering question.
5. Compute the relative effort ratio between two fidelity options using $E(f) = E_0 \cdot k^f$ and use the result to justify a fidelity choice.

3.1 Prototype = Answer to a Question, Reducer of Risk

A prototype is not a small version of the final product. It is not a demonstration for a trade show, a proof that your team is making progress, or practice for the real build. A prototype is a **minimum sufficient artifact** constructed to answer one specific technical or business question and to retire a corresponding risk.

Write the question down before you pick up a soldering iron. If you cannot write it in a single sentence, you do not yet know what you are building. Consider the difference between these two framings:

- *Vague:* "We need to build a prototype of the sorter."
- *Precise:* "Can a sub-$500 CAD BOM sorter module achieve $\geq$95 % sort accuracy at 6 items/min with reliable MQTT connectivity over standard Wi-Fi?"

The precise framing tells you exactly what to build (a low-cost sorter mechanism with Wi-Fi MQTT), what to measure (accuracy at a fixed throughput rate), and what threshold constitutes a "yes" answer. It also tells you what *not* to build: an injection-moulded housing, CE-marked power supply, or production PCB layout are irrelevant until the accuracy and connectivity question is answered.

Risk in this context is the probability that an unverified assumption collapses the project. Every assumption you have not tested is a risk. A prototype converts an assumption into a measurement, and a measurement into a decision. Figure 3.1 shows how even a low-fidelity prototype delivers substantial intrinsic value — knowledge and risk reduction — long before artifact quality approaches production standards.

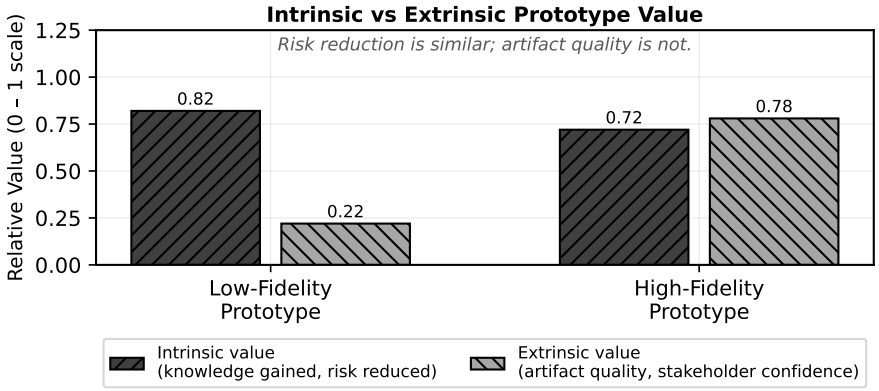

Figure 3.1: Intrinsic value (knowledge, risk reduction) is substantial even at low fidelity; extrinsic value (artifact quality, stakeholder confidence) grows mainly at high fidelity. Build to the fidelity the question requires, not the fidelity that impresses.

A prototype delivers two kinds of value. **Intrinsic value** is the knowledge gained and the risk retired by answering the central question; it is high even at low fidelity, because the question does not require a polished artifact. **Extrinsic value** is the quality and completeness of the artifact itself — its appearance, robustness, and the confidence it gives to external stakeholders. Extrinsic value grows mainly at higher fidelity levels because it depends on how finished the prototype looks and

behaves, not on whether the central question is answered. Build to the fidelity that delivers the intrinsic value you need; extrinsic value follows in later milestones.

Framing every build as an answer to a question keeps the team aligned and prevents the most common prototyping failure: building features before the central question is answered.

3.2 The Seven Stages

Figure 3.2 shows the full Prototype Development Cycle. The seven stages run left to right; Stages 4 through 6 form the core build-test loop that iterates until the central question is answered. The loop is controlled — you exit it on a criterion, not when you run out of time.

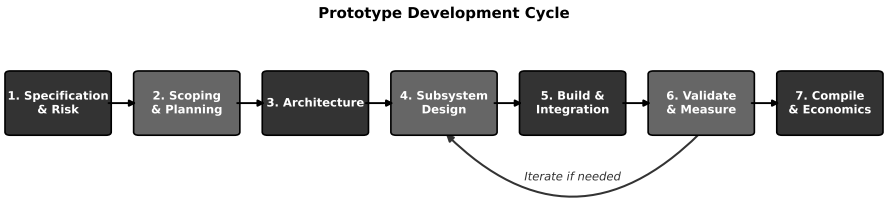

Figure 3.2: The seven-stage Prototype Development Cycle with the Design ↔ Validate iteration loop. Stages 4 through 6 form the core build-test loop; iteration is controlled, not ad-hoc.

3.2.1 Stage 1: Specification & Risk

Stage 1 transforms the project brief into a structured set of **specifications** and a ranked list of **risks**. You are not designing anything yet. You are reading, asking questions, and writing.

A specification at this stage has four components:

1. *Functional requirements* — what the system must do ("sort at $\geq$6 items/min").
2. *Performance bounds* — measurable thresholds ("$\geq$95 % accuracy", "MQTT round-trip $\leq$200 ms").
3. *Constraints* — non-negotiable limits ("BOM cost $\leq$$500 CAD", "fits on a 600 mm bench conveyor").
4. *Assumptions* — things believed to be true but not yet measured.

Every assumption in item 4 is a risk candidate. You rank risks by the product of probability and impact, and you identify which prototype stage will retire each one. By the end of Stage 1 you have a written specification document and a risk register. These are the two artifacts that every subsequent stage references.

3.2.1.0.1 In Practice. A one-hour specification session with your full team at the whiteboard saves more time than a week of uncoordinated building. Force every team member to write at least one assumption. Assumptions that no one writes down are the ones that kill projects in Week 10.

3.2.2 Stage 2: Scoping & Planning

Stage 2 converts the risk register into a prototype plan. You decide: which risks must be retired by this prototype, what fidelity level is sufficient to retire them, how long the build will take, and what success looks like.

The key output is a **prototype plan** with:

- The central question (one sentence).
- The chosen fidelity level and justification.
- A work-breakdown with task ownership and duration.
- Explicit pass/fail criteria for the validation step.

Scoping means saying "not this prototype." If the question is about sort accuracy, the housing material is out of scope. Resist the urge to answer every question at once; you will get a chance in the next milestone. A well-scoped prototype plan is a document your supervisor can review in ten minutes and approve or correct before your team spends any build time — and it is the key output that justifies a Gate 2 approval; Chapter 4 develops it into a full specification and risk register.

3.2.3 Stage 3: Architecture

Stage 3 partitions the system into **subsystems** with defined interfaces. You are not selecting components yet; you are drawing boundaries.

A useful architecture diagram shows:

- Each subsystem as a labelled block.
- Every signal and power interface as an arrow with direction and type (digital, analog, mechanical, protocol name).
- The measurement points where validation data will be collected.

The architecture is the contract between team members. If two people are building adjacent subsystems without an agreed interface, integration will fail. In Stage 3 you negotiate and document those contracts before anyone starts detailed design.

The architecture also determines which subsystem risks can be tested independently. If sort accuracy depends on both the sensor subsystem and the conveyor speed controller, you cannot test one without the other — and your plan must reflect that coupling.

3.2.4 Stage 4: Subsystem Design

Stage 4 is detailed design: component selection, schematic capture, firmware architecture, mechanical part design, and BOM costing. This is where your prior coursework in PCB design, embedded programming, and mechanical CAD becomes the primary activity.

In a prototype context, Stage 4 output does not need to be production-ready. A breadboard schematic is a legitimate Stage 4 artifact if it enables the Stage 5 build to proceed and the Stage 6 measurement to be valid. Keep every design decision traceable to a specification line or a risk item. If you cannot trace it, question whether you need it.

Stage 4 is the entry point to the Design ↔ Validate loop described in Section 3.3. You design at the subsystem level, build the subsystem, measure it, and return to design if the measurement fails — before integrating with adjacent subsystems.

3.2.5 Stage 5: Build & Integration

Stage 5 is fabrication and assembly: soldering, 3D printing, firmware flashing, wiring, and bringing subsystems together at their defined interfaces.

Integration is a distinct activity from subsystem build. Subsystems that each pass their individual tests routinely fail at integration because interface assumptions were wrong. Run integration in a defined order: bring up the simplest subsystem first, verify its standalone behaviour, then attach the next subsystem and re-verify the combined behaviour before adding the third.

Document every deviation from the Stage 4 design as it happens; post-hoc reconstruction from memory is unreliable and undermines the entire design stage.

3.2.5.0.1 In Practice. Keep a shared build log — a shared document or a lab notebook photographed daily — recording what was built, what changed from the design, and any anomalies observed. The build log is the evidence that your Stage 6 validation measurements are trustworthy.

3.2.6 Stage 6: Validate & Measure

Stage 6 is where you answer the question you wrote in Stage 2. **Validation** is a structured measurement campaign against the pass/fail criteria defined before you built anything.

A valid measurement has:

- A defined test procedure that a second engineer can reproduce.
- Sufficient repetitions to estimate uncertainty (minimum 10 trials for a binary outcome like "sort correct/incorrect").

- Documented environmental conditions (supply voltage, temperature, Wi-Fi signal strength, conveyor belt load).
- A recorded outcome: pass, fail, or inconclusive with reason.

If the outcome is "pass," you exit the loop and proceed to Stage 7. If the outcome is "fail," you return to Stage 4 with a specific design change hypothesis, not a vague intention to "try something different." This controlled return is what distinguishes disciplined iteration from thrashing.

3.2.7 Stage 7: Compile & Economics

Stage 7 closes the prototype milestone. You compile the evidence: test results, BOM cost actuals, build time log, and a comparison of achieved performance against specification. You then assess the economic implications.

For a commercial project, Stage 7 includes a preliminary **unit economics** analysis: does the validated BOM cost, at the production volume assumed in the business model, support the target margin? For the running example — a 12-person start-up with a Hardware-as-a-Service model, price $p = \$1{,}200$ CAD, and variable cost $c_{\text{var}} = \$480$ CAD — the contribution margin per unit is:

$$m = p - c_{\text{var}} = \$1{,}200 - \$480 = \$720 \text{ CAD} \tag{3.1}$$

If the BOM comes in above $500 CAD target, Stage 7 flags that the economics are compromised before the team invests in Milestone 2 tooling. Stage 7 also produces the **go/no-go recommendation** for the next milestone and a list of risks that remain open. This recommendation is the prototype-evidence component of the Gate 3 decision: the project proceeds to Stage 3 development only if the Stage 7 compilation supports it.

3.3 The Controlled Iteration Loop (Design $\leftrightarrow$ Validate)

Stages 4, 5, and 6 form a loop. The loop is necessary because prototypes reveal information that design alone cannot predict: a sensor saturates at lower ambient light than the datasheet implied, firmware interrupt latency is 40% higher than estimated, a 3D-printed bracket flexes under motor torque. Iteration is not a sign of poor planning; it is the mechanism by which prototyping generates knowledge.

What must be controlled is *how* you iterate.

Uncontrolled iteration looks like this: the test fails, someone changes three things simultaneously, the next test passes, and no one knows which change fixed the problem. This is common; it is also useless as engineering knowledge, because you cannot reproduce the fix reliably or explain it to a manufacturing partner.

Controlled iteration imposes two rules:

1. *One variable at a time.* When a measurement fails, form a hypothesis about the cause, change exactly one design element, and re-measure. If you must

change two things to make the system testable, document the dependency explicitly.

2. *Exit on a criterion, not on a deadline.* Define in Stage 2 what "good enough" means. Exit the loop when the measurement meets that criterion. Do not exit because the schedule says so without recording which criteria remain unmet.

Every pass through the loop should reduce uncertainty by answering a smaller sub-question. If you cannot name the sub-question before you change something, you are not iterating — you are guessing.

3.3.0.0.1 In Practice. Limit yourself to three full loop passes before calling a team checkpoint. If after three passes the central question is not answered, the problem is likely in the scope, the architecture, or the specification — not in the build. Step back to Stage 2 or Stage 3 rather than continuing to iterate at Stage 4.

3.4 Prototype Types

Not all prototypes serve the same purpose. Selecting the right type for your question prevents over-building and keeps the team focused. The four primary types used in engineering product development are:

Proof-of-Concept (PoC) Answers a binary feasibility question: can the physics work? Appearance is irrelevant. A PoC uses whatever materials are fastest to assemble — breadboards, hook-up wire, 3D-printed brackets, off-the-shelf modules. Target fidelity: Level 1 or 2.

Functional prototype Demonstrates that the system performs its core function in representative conditions. Appearance may still be a mock-up. Firmware is real but not hardened. Used to answer "does it work under realistic operating conditions?" Target fidelity: Level 3.

Looks-like/works-like prototype Combines representative form and full function. Housing geometry, interface locations, and weight approach final product. Used to answer "is the user experience acceptable?" and "can we hit the cost target?" Target fidelity: Level 4.

Pre-production prototype Built from production-intent processes: PCB from the chosen contract manufacturer, housing from production tooling, firmware at release candidate version. Answers: "will this survive the factory and the field?" Target fidelity: Level 5.

Choosing a looks-like/works-like prototype to answer a feasibility question is the most common and most expensive mistake in early-stage product development. Match the type to the question first; fidelity follows from type.

3.5 Fidelity Matched to the Question

Fidelity is the degree to which a prototype resembles and performs like the final product. Higher fidelity costs more. The cost growth is not linear.

Figure 3.3 shows relative effort plotted against fidelity level. The relationship is approximately exponential.

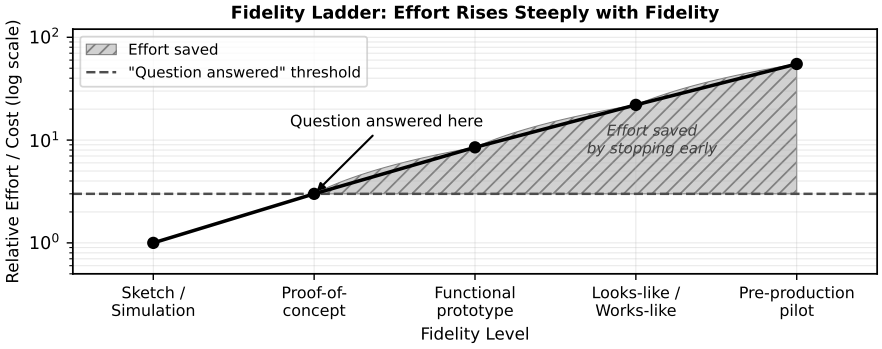

Figure 3.3: Relative effort rises steeply with fidelity level. A proof-of-concept (Level 2) answers many questions at a fraction of the cost of a looks-like/works-like prototype (Level 4).

3.5.1 The Effort–Fidelity Model

Let $f \in \{1, 2, 3, 4, 5\}$ be the fidelity level, where $f = 1$ is a hand sketch or simulation and $f = 5$ is a pre-production prototype built with production-intent processes. Define **relative effort** $E(f)$ as:

$$E(f) = E_0 \cdot k^f, \qquad k > 1 \tag{3.2}$$

where E_0 is a baseline effort constant and k is the per-level effort multiplier. The model captures the empirical observation that each fidelity step requires roughly k times the total effort of the step below it, because higher fidelity demands tighter tolerances, more sophisticated tooling, more thorough testing, and more documentation.

3.5.1.1 Effort Table

Set $E_0 = 1$ and $k = 2.5$ as representative values. Table 3.1 shows the resulting effort at each level.

The ratio of effort at $f = 5$ relative to $f = 2$ is:

$$\frac{E(5)}{E(2)} = \frac{k^5}{k^2} = k^3 = 2.5^3 = 15.6 \tag{3.3}$$

If your question is answered at $f = 2$, building to $f = 5$ multiplies your effort by $15.6\times$. Equivalently, $\frac{E(5)-E(2)}{E(5)} \approx 93\,\%$ of the total $f = 5$ effort is *spent after the question is already answered*. That is the waste the fidelity discipline prevents.

Table 3.1: Relative effort $E(f) = 2.5^f$ for fidelity levels 1 through 5.

Level f	**Type**	**Relative Effort** $E(f)$
1	Sketch / simulation	$2.5^1 = 2.5$
2	Proof-of-concept	$2.5^2 = 6.25$
3	Functional prototype	$2.5^3 = 15.6$
4	Looks-like/works-like	$2.5^4 = 39.1$
5	Pre-production	$2.5^5 = 97.7$

3.5.2 Running Example: Choosing Fidelity for the Sorter

Return to the central question for the connected IoT robotic sorting subsystem:

> "Can a sub-$500 CAD BOM sorter module achieve $\geq$95 % sort accuracy at 6 items/min with reliable MQTT connectivity over standard Wi-Fi?"

The team is deciding between two prototype options:

Option A — Looks-like/works-like ($f = 4$): Custom PCB with production layout, injection-moulded housing mock-up, full production firmware with over-the-air update support. Estimated build time: 14 weeks.

Option B — Proof-of-concept ($f = 2$): Breadboard electronics, 3D-printed mounting bracket, firmware with sort accuracy event logging, Wi-Fi MQTT connectivity test. Estimated build time: 3 weeks.

Applying the effort model with $k = 2.5$:

$$E(4) = k^4 = 2.5^4 = 39.1 \quad \text{(Option A effort)}$$
$$E(2) = k^2 = 2.5^2 = 6.25 \quad \text{(Option B effort)}$$

$$\frac{E(4)}{E(2)} = k^{4-2} = 2.5^2 = 6.25 \tag{3.4}$$

The fraction of Option A's effort consumed *after* the question is already answerable at Level 2 is:

$$\frac{E(4) - E(2)}{E(4)} = \frac{39.1 - 6.25}{39.1} \approx 0.84 \quad (84\,\%). \tag{3.5}$$

If a team's build estimate is T weeks for Option A, Option B requires $T \times E(2)/E(4) = T/6.25$ weeks. For the 14-week Option A estimate: $14/6.25 \approx 2.2$ weeks — close to the stated 3-week Option B estimate (the difference reflects real project overhead not captured by the model).

Option A requires $6.25\times$ the effort of Option B. Now ask: does the central question require $f = 4$? The question asks about sort accuracy, throughput rate, BOM cost, and MQTT connectivity. None of those require an injection-moulded housing or a production PCB. A breadboard can deliver accurate sensor readings; a 3D-printed bracket can hold the actuator at the required geometry; a Wi-Fi module on a breadboard can carry MQTT traffic.

The decision is Option B. If Option B answers "yes," the team proceeds to Milestone 2 with the validated architecture, and Option A becomes the correct investment at that stage. If Option B answers "no" — the sort accuracy is 78% at 6 items/min, or the MQTT latency is unacceptable on a congested network — the team has lost 3 weeks, not 14. The cost of a wrong hypothesis is bounded by the fidelity choice.

3.6 The Documentation Layer

Every stage of the Prototype Development Cycle produces a document. This is not administrative overhead — it is the mechanism by which the prototype generates knowledge that survives beyond the build. A result that is not recorded is a result that will be repeated, at full cost, by the next engineer who faces the same question.

Stage 1 produces the specification and risk register. **Stage 2** produces the prototype plan with pass/fail criteria. **Stage 3** produces the architecture diagram and interface table. **Stage 4** produces schematics, firmware architecture notes, and the BOM; any deviation from the plan is noted here. **Stage 5** produces the build log: what was assembled, what changed from Stage 4, and photographs of key assembly steps. **Stage 6** produces the test procedure, raw data table, and the pass/fail verdict with reasoning. **Stage 7** produces the portfolio entry: test results, cost actuals, and the go/no-go recommendation.

The documentation is not assembled at the end of the milestone. It is produced at each stage, incrementally, so that the portfolio at Stage 7 requires only compilation, not reconstruction from memory. If you find yourself writing Stage 1 documentation in Week 11, the process has already failed. The portfolio you submit at Milestone 3 is exactly the set of documents you produced at each stage — nothing more, nothing less.

3.6.0.0.1 Common Pitfalls.

- **"Prototype = mini-product."** Building all product features when only one needs to be proved is the most common and most expensive prototyping error. Injection moulding, CE certification, and production firmware are irrelevant at Stage 4 if the question is about sort accuracy. Every feature you build beyond what the question requires is pure waste until the question is answered.
- **"Higher fidelity is always better."** Fidelity costs effort exponentially, as shown in Table 3.1. A higher-fidelity prototype that cannot yet answer the question delivers zero additional information — it only consumes schedule and budget. Match fidelity to the question; resist the instinct to impress.

Review Questions

1. **(Conceptual)** List the seven stages of the Prototype Development Cycle in order. For Stage 1 and Stage 6, state the primary output that stage produces.

2. **(Conceptual)** Distinguish **intrinsic value** from **extrinsic value** in the context of a prototype. Which type does a proof-of-concept maximise, and why does that make it appropriate for answering a central technical question?
3. **(Applied — calculation)** Using $E(f) = E_0 \cdot k^f$ with $E_0 = 1$ and $k = 3.0$:
   (a) Compute $E(f)$ for $f = 1, 2, 3, 4, 5$.
   (b) What is the ratio $E(4)/E(2)$?
   (c) If a question is answered at $f = 2$, what fraction of the Level-4 effort is saved?
4. **(Applied — decision)** A team must verify that a custom MEMS pressure sensor achieves ± 0.5 Pa accuracy when integrated with their MCU at $-20\,^{\circ}\text{C}$.
   (a) Identify the most appropriate prototype type from §3.4.
   (b) State the fidelity level from §3.5 and justify the choice in one sentence.
   (c) (c) If the PoC result is inconclusive — accuracy is within spec at some temperatures but not others — describe one controlled iteration step you would take at Stage 4 before changing the fidelity level.
5. **(Applied — multi-step)** A team estimates 20 weeks to build a pre-production prototype ($f = 5$) at a weekly cost of $3,500 CAD. Using $k = 2.5$, $E_0 = 1$:
   (a) Compute $E(2)$ and $E(5)$.
   (b) What fraction of the 20-week effort corresponds to $f = 2$?
   (c) How many weeks does the PoC take (round to one decimal place)?
   (d) How much money is saved by stopping at $f = 2$ if the question is answered there?

Try It Yourself

A firmware team must choose between two prototype options to validate their custom RTOS scheduler under worst-case interrupt load. Both options use $E_0 = 1$ and $k = 3.0$.

Option X — Proof-of-concept ($f = 2$): A minimal test harness running the scheduler on target hardware with synthetic interrupt injection. Estimated build time: 2 weeks.

Option Y — Functional prototype ($f = 3$): Full firmware stack with all application tasks active, running the complete RTOS configuration. Estimated build time: 9 weeks.

1. Compute $E(2)$ and $E(3)$ using $E(f) = E_0 \cdot k^f$.
2. Find the effort ratio $E(3)/E(2)$.
3. Compute the fraction of Option Y effort consumed after the question is already answerable at $f = 2$ (use the formula from Equation 3.5).
4. The central question is: "Does the scheduler meet the 1 ms worst-case interrupt latency under full task load?" State in one sentence whether Option X or Option Y is the correct choice and why, referencing the effort ratio.

(No answer key provided.)

End-of-Chapter Artifact

The deliverable for this chapter is a single written paragraph, submitted to your portfolio, stating: (1) the central question your Milestone 1 prototype will answer, (2) the fidelity level you have chosen, and (3) a justification for that choice in terms of the question, the risk it retires, and the effort saved relative to the next higher fidelity level. This statement is your Milestone 1 prototype scope; submit it as the prototype-evidence component of your Gate 2 package in Week 4.

Running-Example Statement (model your own on this):

> The Milestone 1 prototype for the IoT robotic sorting subsystem will answer the following central question: *"Can a sub-$500 CAD BOM sorter module achieve $\geq$95% sort accuracy at 6 items/min with reliable MQTT connectivity over standard Wi-Fi?"* The chosen fidelity is Level 2 (proof-of-concept), implemented as breadboard electronics, a 3D-printed mechanical mounting bracket, and firmware with sort-event data logging and Wi-Fi MQTT connectivity. This fidelity is sufficient because the question is about sensor accuracy, actuator throughput, and network protocol performance — none of which require a production PCB layout or an enclosure. Advancing to Level 4 at this stage would multiply effort by $6.25\times$ (Equation 3.4) before the feasibility of the core mechanism is established. If the Level 2 prototype answers "yes," Level 4 is the justified next investment; if "no," the team reframes the architecture having lost 3 weeks rather than 14.

This statement feeds directly into the specification document you will develop in Chapter 4. Keep it in your portfolio; you will reference and refine it at each subsequent milestone.

Chapter Summary

A prototype is the minimum sufficient artifact that answers a specific technical or business question and retires a corresponding risk. The Prototype Development Cycle organises that work into seven stages: Stage 1 (Specification & Risk) establishes what must be true and what is assumed; Stage 2 (Scoping & Planning) sets the question, the fidelity, and the success criterion; Stage 3 (Architecture) partitions the system into subsystems with defined interfaces; Stage 4 (Subsystem Design) produces the design artefacts; Stage 5 (Build & Integration) realises and assembles those designs; Stage 6 (Validate & Measure) answers the question against predefined criteria; and Stage 7 (Compile & Economics) closes the milestone with evidence and a go/no-go recommendation.

Stages 4 through 6 form a controlled iteration loop. Each pass tests one hypothesis, changes one variable, and exits on a measured criterion — not on a deadline and not by accumulated intuition.

Fidelity is not a virtue. It is a variable you set to the minimum level that makes the central question answerable. The effort–fidelity relationship is exponential ($E(f) =$

$E_0 \cdot k^f$), so every unnecessary fidelity level you add multiplies your cost by k before you have an answer. For the sorter running example, choosing Level 2 over Level 4 saves $6.25\times$ the effort on a question whose answer is not yet known.

Transition to Chapter 4. Chapter 4 turns the central question you have just defined into a complete specification and risk register. You will write measurable, testable requirements, separate "what" from "how," and rank risks using the FMEA risk-priority number — producing the core deliverable of Stage-Gate Stage 2 and the foundation for every subsequent design decision.

CHAPTER 4

SPECIFICATIONS & RISK PRIORITIZATION

4.0.0.0.1 Where this fits. This chapter produces the core deliverable of Stage-Gate Stage 2 (Scoping) — the product specification and risk register that justify a Go decision at Gate 2. The central question and fidelity statement you produced in Chapter 3 define the *goal*; the specification turns that goal into measurable, testable requirements, and the risk register ranks the unknowns the prototype must resolve before Gate 2 can be passed.

The central question you defined in Chapter 3 identifies the goal; this chapter supplies the standard for success — converting that goal into measurable, testable requirements and a ranked risk register using two complementary tools: a simple risk equation and an FMEA Risk Priority Number.

By the end of this chapter you will be able to:

1. write measurable, testable requirements that can unambiguously pass or fail;
2. separate requirements (the *what*) from design decisions (the *how*);
3. build and defend a risk-priority ranking that guides prototype focus; and
4. identify the critical unknown in a set of open assumptions and justify which prototype experiment must be run first.

4.1 Anatomy of a Specification

A specification is a contract between the engineering team and every stakeholder who must eventually verify the product. It answers one question per line: *how will we know we succeeded?* A well-formed specification document contains five distinct element types, each serving a different verification role.

4.1.1 Functional Requirements

A functional requirement states a capability the system must provide, expressed as a verb phrase with a measurable threshold. The threshold is what separates a functional requirement from a vague aspiration.

- **Weak (untestable):** "The system shall sort items quickly."
- **Strong (testable):** "The system shall classify and divert each item within $\leq$167 ms of entering the sensor gate (throughput $\geq$6 items/min, continuous 1-hour run)."

Every functional requirement must name: (a) the actor or subsystem, (b) the action, (c) the measurable threshold, and (d) the operating condition under which the threshold applies.

4.1.2 Performance Targets

Performance targets quantify quality-of-behaviour: accuracy, latency, bandwidth, efficiency. They differ from functional requirements in that they govern *how well* the function is performed, not merely *whether* it is performed. A sorter that classifies items at 6 items/min but with 40% error satisfies the throughput functional requirement yet fails the accuracy performance target.

4.1.3 Constraints

Constraints are non-negotiable boundaries imposed by physics, regulation, or business. They do not describe desirable behaviour; they eliminate design options outright. Examples: BOM cost $\leq$\$500 CAD (business constraint), supply voltage from USB $\leq$5 V at 2 A (interface constraint), operating temperature 0–50°C (environmental constraint).

4.1.4 Interfaces

Interface specifications describe the boundaries between subsystems, between the product and external systems, and between the product and its users. A mechanical interface names connector types and physical envelope; an electrical interface names voltage levels, connector pinouts, and communication protocols; a software interface names message formats, topics, and timing. Specifying interfaces early prevents integration surprises — the most common source of late-stage schedule overrun.

4.1.5 Acceptance Criteria

Acceptance criteria translate every requirement into a concrete test: apparatus, procedure, pass/fail decision rule. Writing the acceptance test at the same time as the requirement is a discipline check: if you cannot describe how to measure it, the requirement is not yet measurable.

4.1.5.0.1 In Practice. At Vanier's Product Development Lab, teams have learned to draft acceptance criteria as a three-column table: *Requirement ID*, *Test Procedure* (apparatus + steps), *Pass Condition*. Filling this table during Week 2 surfaces ambiguous requirements before any hardware is ordered, saving the cost of a redesign sprint.

4.2 Requirements vs. Design

The single most persistent error in student (and professional) specification documents is embedding a design decision inside a requirement. Figure 4.1 shows a side-by-side comparison of requirement and design statements for the same capability.

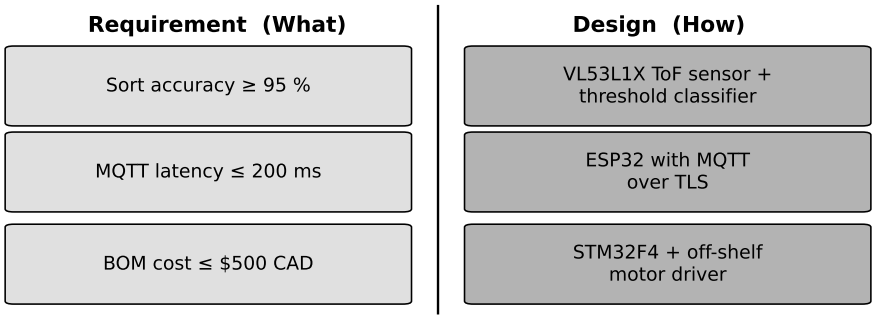

Figure 4.1: Side-by-side comparison of a requirement statement (left) and a design statement (right) for the same capability. Requirements describe observable, measurable outcomes; design statements prescribe implementation choices.

A useful test: replace the specific implementation with an alternative one. If the requirement is still meaningful, it is a genuine requirement. If it becomes nonsensical, it was a design decision dressed up as a requirement.

- **Requirement:** "Classification results shall be transmitted to the cloud dashboard within $\leq$200 ms of each sort event." Replacing Wi-Fi with Ethernet or cellular still leaves this requirement meaningful. ✓
- **Design-embedded (invalid):** "The MCU shall use MQTT over Wi-Fi to transmit classification results." This mandates the implementation, closing off alternatives before the team has evaluated trade-offs. ×

4.2.1 The "What / How" Discipline

Every line of a specification document should survive the question: "Does this describe what the system must do, or does it describe how we plan to do it?" The *what* belongs in the specification. The *how* belongs in the design document, the architecture diagram, or the BOM — produced *after* requirements are frozen.

4.2.1.0.1 In Practice. When reviewing a teammate's specification draft, highlight every noun that names a specific component ("Arduino", "ToF sensor", "Python")

or a specific topology ("mesh network", "I^2C bus"). Each highlighted phrase is a candidate design decision that may be pre-empting a better alternative. Move it to the design rationale section and restate the requirement in terms of observable behaviour.

4.2.1.0.2 Common Pitfalls.

- **Unmeasurable criteria.** "The system shall be user-friendly" cannot be tested. Restate as: "A first-time operator shall complete initial setup in $\leq$10 minutes following the Quick-Start card, with zero support calls, measured on five naïve participants."
- **Embedding the solution.** "The sensor shall use a VL53L1X time-of-flight module" is a design choice. The requirement should read: "The sensor subsystem shall detect items up to 300 mm from the gate with $\leq$2 mm resolution at throughput $\geq$6 items/min."
- **Ambiguous conditions.** A requirement with no stated operating condition is incomplete. "Latency $\leq$200 ms" must specify: under what network load? at what ambient temperature? for what message size?

4.3 Risk-Driven Prioritization

Once requirements exist, the team must decide which risks — failures to meet those requirements — deserve the most engineering attention. Without a principled ranking, you either chase the most visible problem rather than the most dangerous one, or try to mitigate everything simultaneously and mitigate nothing well.

4.3.1 The Basic Risk Equation

The simplest risk model assigns two quantities to each potential failure mode:

- P_{failure}: the estimated probability that the failure occurs in a prototype or field unit, scored 1–10 (1 = very unlikely, 10 = near-certain).
- $C_{\text{consequence}}$: the estimated cost of that failure — in schedule days, dollars, or safety impact — scored 1–10 (1 = trivial, 10 = project-ending).

$$R = P_{\text{failure}} \times C_{\text{consequence}} \tag{4.1}$$

The resulting risk score R ranges from 1 to 100. Teams typically set a threshold (e.g., $R \geq 30$) above which a mitigation plan is mandatory. Figure 4.2 shows a probability–consequence matrix that maps risk scores to action zones.

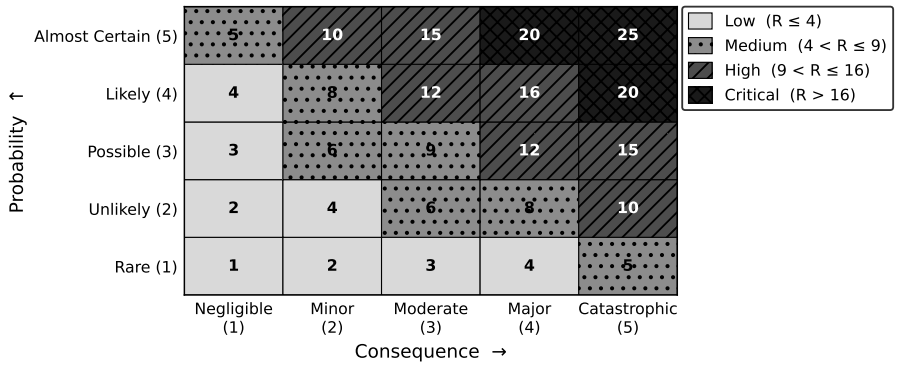

Figure 4.2: Probability $\times$ consequence risk matrix for the running example. Cells with $R \geq 30$ fall in the red action zone; $15 \leq R < 30$ in the amber watch zone; $R < 15$ in the green monitor zone.

4.3.2 Limitations of the Two-Factor Model

Equation (4.1) is fast to communicate but conflates two distinct ideas: the probability that a failure occurs and the team's ability to detect it before it reaches the user. Section 4.4 adds detection as a third factor.

4.4 FMEA-Style Ranking (RPN)

Failure Mode and Effects Analysis (FMEA) is a structured method for systematically enumerating every way a system can fail, estimating three quantities for each failure mode, and computing a Risk Priority Number to guide mitigation effort.

4.4.1 The RPN Formula

$$\mathrm{RPN} = S \cdot O \cdot D \tag{4.2}$$

where:

- S = Severity (1–10): how bad is the consequence if the failure reaches the user?
- O = Occurrence (1–10): how likely is the failure to occur during normal operation?
- D = Detection (1–10): how difficult is it to detect the failure before it affects the user? *Note: a higher D score means harder to detect.*

RPN ranges from 1 to 1,000. Unlike the two-factor risk score R, the RPN captures the value of test coverage: a team that adds a robust bench test lowers D and therefore lowers RPN even without changing the underlying design.

4.4.1.0.1 In Practice. Industry FMEA tables typically flag RPN ≥ 100 as requiring a documented corrective action, and RPN ≥ 200 as requiring a design change before release. For a student prototype, a lighter threshold — flag RPN ≥ 80 — is appropriate, because prototype runs are short and failures are recoverable. These are the failure modes you must address before your Gate 2 submission.

4.4.2 Scale Anchors

Table 4.1 gives representative scale anchors for S, O, and D as used in this course. Teams should agree on anchors *before* scoring to prevent rater drift.

Table 4.1: Representative 10-point scale anchors for FMEA scoring in a prototype context.

Score	Severity (S)	Occurrence (O)	Detection (D)
1–2	No user impact	Rare ($<$1 in 1,000 runs)	Immediately obvious; alarms fire
3–4	Minor degradation	Occasional ($\sim$1 in 100 runs)	Detectable by inspection or log
5–6	Partial loss of function	Moderate ($\sim$1 in 20 runs)	Detectable only with test setup
7–8	Major loss of function	Frequent ($\sim$1 in 5 runs)	Difficult; requires special tool
9–10	Safety / total failure	Near-certain ($>$1 in 2 runs)	Cannot be detected before impact

4.5 The Critical Path of Unknowns

The risk register produced by FMEA answers *which* risks are largest. A separate but related question is *which risks must be resolved first* — because some unknowns block progress on every other subsystem. This is the critical path of unknowns.

Figure 4.3 illustrates the assumption list and dependency arrows for the running example. The highest-RPN item that *also* blocks downstream work defines the prototype's first experiment.

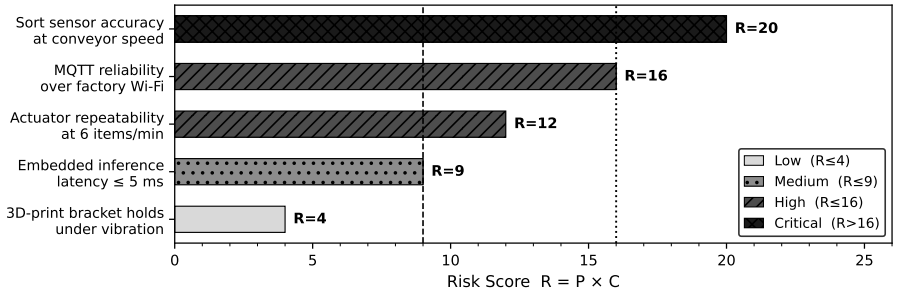

Figure 4.3: Critical unknowns and assumption dependency map for the IoT robotic sorting subsystem. Arrows show which assumptions must be resolved before dependent subsystem work can begin. The sort-sensor assumption (shaded) lies on the critical path because both the divert actuator timing and the firmware logic depend on its output format.

4.5.1 Identifying the Critical Unknown

To find the critical unknown, list every open assumption in the risk register and ask two questions for each:

1. If this assumption turns out to be false, how many other subsystems must change? (Coupling score.)
2. How quickly can we resolve it with a targeted experiment? (Resolvability score.)

An assumption with high coupling and low resolvability is both dangerous and slow to clear — it should head the prototype schedule. An assumption with high coupling but high resolvability should be the *first* experiment run, because a quick test removes the blocker cheaply.

4.5.1.0.1 In Practice. For the IoT sorter, the sort-sensor detection principle (time-of-flight vs. break-beam vs. vision) is the critical unknown: the firmware state machine, the actuator timing, and the MQTT payload format all depend on what the sensor outputs. Running a 48-hour bench test of candidate sensors *before* ordering actuator parts avoids a costly mid-sprint pivot.

4.6 Worked Example: IoT Robotic Sorting Subsystem

This section walks through the full specification and risk-ranking workflow for the running example established in Chapter 3: a connected IoT robotic sorting subsystem classifying and diverting items on a bench conveyor, reporting over MQTT/Wi-Fi, with a target BOM of $\leq$\$500 CAD and a sort accuracy of $\geq$95%.

4.6.1 Step 1: Convert Vague Wishes into Measurable Requirements

The team's initial wish list contained five phrases typical of early ideation. Table 4.2 shows each vague phrase converted to a testable requirement.

The most instructive conversion is "fast":

- **Vague:** "The system shall process sensor data fast."
- **Step 1 — name the quantity:** Processing speed is measured in samples per second (kS/s) and as end-to-end latency (ms).
- **Step 2 — anchor to the use case:** At 6 items/min, one item arrives every $60\,\text{s}/6 = 10\,\text{s}$. A single sort event reads one sensor frame, runs the classifier, and commands the actuator. The conveyor belt speed is 50 mm/s and the item window at the gate is 50 mm, giving a gate dwell time of $50\,\text{mm} \div 50\,\text{mm/s} = 1{,}000\,\text{ms}$. The actuator must fire within that window.
- **Step 3 — allocate the budget:** Budget allocation: sensor read 1 ms, classifier 2 ms, actuator command 2 ms, margin 995 ms.

$$t_{\text{firmware}} = 1\,\text{ms} + 2\,\text{ms} + 2\,\text{ms} = 5\,\text{ms}. \tag{4.3}$$

The remaining $1{,}000 - 5 = 995\,\text{ms}$ is dwell-time margin before the next item enters the gate. Conservatively, the end-to-end firmware latency budget is

$\leq$5 ms. At a 10 kS/s ADC rate, the sensor delivers $10{,}000$ samples per second; one classification frame uses 10 samples, costing $10/10{,}000 = 1\,\text{ms}$ — within budget.

- **Measurable requirement:** "The MCU firmware shall process one classification event (sensor read + classification + actuator command) within $\leq$5 ms, measured from sensor-trigger interrupt to actuator-enable GPIO, at a sustained rate of $\geq$10 kS/s sensor sampling."

Table 4.2: Vague feature wishes converted to measurable, testable requirements for the IoT robotic sorting subsystem. Each requirement identifies actor, action, threshold, and operating condition.

Vague Wish	Measurable Requirement	ID
"Sorts accurately"	The system shall classify items with $\geq$95% accuracy (correct class / total items) over a continuous 100-item run at nominal belt speed.	REQ-01
"Fast"	The MCU firmware shall complete one classification event within $\leq$5 ms (sensor read + classify + actuator command) at $\geq$10 kS/s sensor sampling.	REQ-02
"Enough throughput"	The system shall sustain $\geq$6 items/min throughput for $\geq$60 min without thermal throttling or error accumulation.	REQ-03
"Sends data to the cloud"	Each sort-event MQTT message shall be delivered to the broker within $\leq$200 ms of the sort event, on a 2.4 GHz Wi-Fi network with $\geq$-70 dBm RSSI.	REQ-04
"Affordable"	The total prototype BOM (all purchased components) shall not exceed $500 CAD.	REQ-05

A sixth implied constraint (power) is added from the hardware context:

CON-01: Total system power draw shall not exceed 5 W when powered from a single USB-C port (5 V, 2 A = 10 W; 5 W provides 50% margin for host compatibility).

4.6.2 Step 2: Compute RPN for Three Subsystems

The team identifies three high-risk subsystems from the Chapter 3 scoping exercise: (A) Sort Sensor, (B) MQTT/Wi-Fi Integration, (C) Divert Actuator. For each, the team identifies the primary failure mode, scores S, O, and D on the 1–10 scale from Table 4.1, and computes RPN using Equation (4.2).

Subsystem A — Sort Sensor

- **Failure mode:** Sensor misclassifies item class (false positive or false negative), causing a sort error.

- **Severity (S):** A miss ships a wrong item — significant product defect, customer complaint, possible recall. $S = 8$.
- **Occurrence (O):** Time-of-flight at <300 mm with a 50 mm/s belt is a moderately proven technique; reflectivity variation across item surface adds uncertainty. $O = 5$.
- **Detection (D):** A classification error is visible in the sort log and confirmed by counting missorted bins at end of run — easily detected during bench test. $D = 2$.
- **RPN:**

$$\text{RPN}_A = S \cdot O \cdot D = 8 \times 5 \times 2 = \mathbf{80}$$

Subsystem B — MQTT/Wi-Fi Integration

- **Failure mode:** MQTT message delivery latency exceeds 200 ms, causing dashboard staleness and potential missed alerts.
- **Severity (S):** Stale dashboard undermines the product's core value proposition (real-time visibility); does not stop physical sorting. $S = 7$.
- **Occurrence (O):** 2.4 GHz Wi-Fi in a lab environment with competing devices experiences frequent retransmissions; MQTT QoS 1 adds handshake overhead. $O = 6$.
- **Detection (D):** Latency can be measured by timestamping publish and subscribe events in the broker log — detectable but requires setup of a monitoring script. $D = 4$.
- **RPN:**

$$\text{RPN}_B = S \cdot O \cdot D = 7 \times 6 \times 4 = \mathbf{168}$$

Subsystem C — Divert Actuator

- **Failure mode:** Actuator fires too late or with insufficient force, failing to divert the item before it exits the gate.
- **Severity (S):** A missed divert is a sort error — same consequence as a sensor misclassification. $S = 8$.
- **Occurrence (O):** Servo or solenoid timing within a 1,000 ms dwell window is well within typical MCU PWM capability; mechanical linkage adds some uncertainty. $O = 3$.
- **Detection (D):** A missed divert is observable by watching the output bins or reading a secondary confirmation sensor — straightforward detection. $D = 2$.
- **RPN:**

$$\text{RPN}_C = S \cdot O \cdot D = 8 \times 3 \times 2 = \mathbf{48}$$

Table 4.3: FMEA Risk Priority Number ranking for the three prototype subsystems of the IoT robotic sorting subsystem. RPN computed using Equation (4.2). Subsystems are sorted by descending RPN.

Rank	Subsystem	Failure Mode	S	O	D	RPN	Action
1	MQTT/Wi-Fi	Latency >200 ms	7	6	4	**168**	Redesign / mitigate
2	Sort Sensor	Misclassification	8	5	2	**80**	Monitor / mitigate
3	Divert Actuator	Late divert	8	3	2	**48**	Monitor

4.6.3 Step 3: Rank and Interpret

Table 4.3 summarises the three RPN scores and maps each to a mitigation action.

Interpretation:

- MQTT/Wi-Fi integration (RPN = 168) is the highest-risk subsystem. The team should prototype and stress-test the Wi-Fi + MQTT stack *first*, before investing time in actuator tuning. Concrete mitigation: run a dedicated latency characterisation script (publish 500 timestamped messages; record broker-side receive timestamp; compute 99th-percentile latency) within Sprint 1.
- Sort Sensor (RPN = 80) exceeds the 80-threshold and requires a documented mitigation plan. The sensor's detection principle should be confirmed in a 100-item accuracy trial before the firmware classifier is written.
- Divert Actuator (RPN = 48) is below the 80-threshold. Standard bench testing during integration is sufficient; no dedicated mitigation sprint is needed.

Applying the basic risk equation (4.1) for cross-check:

$$R_A = P_{\text{failure},A} \times C_{\text{consequence},A} = 5 \times 8 = 40 \quad \text{(action zone)}$$
$$R_B = P_{\text{failure},B} \times C_{\text{consequence},B} = 6 \times 7 = 42 \quad \text{(action zone)}$$
$$R_C = P_{\text{failure},C} \times C_{\text{consequence},C} = 3 \times 8 = 24 \quad \text{(watch zone)}$$

The two-factor scores (4.1) agree with the RPN ranking on the top item (MQTT/Wi-Fi) but obscure the gap between A and B that the detection factor (D) reveals. The sensor is harder to detect as failing, so RPN correctly flags it above the threshold despite its lower occurrence rate.

4.6.4 Try It Yourself

Scenario: A team is developing a **smart irrigation controller** for a rooftop garden. The system reads soil moisture via a capacitive sensor array, decides whether to open a motorised valve, and logs readings to a cloud dashboard over cellular (LTE-M). The initial wish list contains:

- "Waters when needed" — target: soil moisture sensor accuracy $\pm$3% VWC
- "Doesn't over-water" — target: valve response within 30 s of a moisture threshold crossing

- "Cheap to build" — target: BOM $\leq$\$300 CAD

Your tasks:

1. Rewrite each wish as a measurable requirement following the actor–action–threshold–condition format.
2. Identify three failure modes (one per subsystem: moisture sensor, motorised valve, LTE-M uplink).
3. Assign S, O, and D scores on the 1–10 scale with written justification for each S, O, and D value (nine justifications total: three failure modes × three dimensions).
4. Compute RPN for each failure mode using Equation (4.2), showing every multiplication step.
5. Rank the three failure modes from highest to lowest RPN and state which subsystem should be prototyped first.
6. For this cross-check, use the O score as P_{failure} and the S score as $C_{\text{consequence}}$, consistent with the worked example convention. Cross-check your top-ranked item using Equation (4.1) and comment on whether the two rankings agree.

Do **not** use the same S, O, D values as the worked example.

Review Questions

1. **(Conceptual)** Explain the difference between a *functional requirement* and a *performance target*, using one example of each from a product domain of your choice. Why must both be present in a complete specification?
2. **(Conceptual)** A colleague argues that specifying the communication protocol (e.g., "shall use Bluetooth Low Energy") inside a requirement is efficient because it removes ambiguity for the firmware team. Justify why this practice is problematic and describe the correct place in the documentation hierarchy for such a decision.
3. **(Applied — calculation)** A new IoT product team scores three failure modes as follows:

   - Power regulator brown-out: $S = 9$, $O = 2$, $D = 3$
   - Firmware deadlock: $S = 6$, $O = 7$, $D = 8$
   - Enclosure latch failure: $S = 3$, $O = 4$, $D = 1$

   Compute RPN for each failure mode using Equation (4.2), showing every multiplication step. Rank them and state which subsystem requires a documented mitigation plan if the threshold is RPN ≥ 80.
4. **(Applied — multi-step calculation)** A wearable ECG monitor has a motion-artifact failure mode with the following FMEA scores: Severity $S = 8$, Occurrence $O = 4$, Detection $D = 3$.

   (a) Compute $R = P_{\text{failure}} \times C_{\text{consequence}}$ (Equation 4.1) using $P_{\text{failure}} = O = 4$ and $C_{\text{consequence}} = S = 8$ as the cross-check convention. Does this failure mode qualify as high-risk ($R > 16$)?

(b) Compute the RPN. The team then adds a motion-artifact filter that lowers O from 4 to 2 while S and D remain unchanged. Recompute the RPN and state whether the mitigation moves the failure mode below the RPN threshold of 80.

(c) Comment in one sentence on whether the R-score ranking (Part a) and the RPN ranking (Part b) agree.

5. **(Conceptual)** Define the *critical path of unknowns* and explain how it differs from the highest-RPN item in the risk register. Give an example where the highest-RPN item is *not* on the critical path of unknowns and describe what the team should do first.

End-of-Chapter Artifact

You produce the core of **Milestone 1: the Proposal** — together these three parts constitute your Gate 2 evidence package, the scoping deliverables that earn a Go decision to proceed with development. The artifact has three interlocking parts:

1. **Specification Table.** A table of 3–5 measurable requirements (functional requirements + performance targets) and any binding constraints, formatted with columns: ID, Requirement Statement, Threshold, Operating Condition, Acceptance Test. Each row must be testable as written.

2. **Risk Register.** An FMEA table listing at least three failure modes (one per high-risk subsystem identified in Chapter 3). Each row contains: Subsystem, Failure Mode, S, O, D, RPN, Rank, and Mitigation Plan. The table is sorted by descending RPN. Failure modes above RPN 80 must have a specific mitigation action tied to a sprint.

3. **Scoping Rationale.** One paragraph (100–150 words) that names: (a) the central prototype question, (b) the highest-risk assumption (the critical unknown), and (c) the first experiment that will resolve that assumption. This paragraph becomes the executive summary of your Milestone 1 submission.

'What will this thing do, how will you know it works, and what could kill the project?' — these three parts of the Milestone 1 Proposal answer that investor question directly.

Summary

A specification is a contract: every element — functional requirements, performance targets, constraints, interfaces, and acceptance criteria — must be measurable and testable, or it is not yet a requirement. It established the discipline of separating *what* (requirements) from *how* (design), a distinction that keeps early design choices from foreclosing better alternatives. Two complementary risk tools were presented: the two-factor risk equation $R = P_{\text{failure}} \times C_{\text{consequence}}$ (Equation (4.1)) for rapid stakeholder communication, and the FMEA Risk Priority Number RPN $=$

$S \cdot O \cdot D$ (Equation (4.2)) for engineering-level prioritisation that accounts for detection difficulty. Applied to the IoT robotic sorting subsystem, the analysis ranked MQTT/Wi-Fi integration (RPN = 168) as the first prototype target, followed by the sort sensor (RPN = 80), and confirmed that the divert actuator (RPN = 48) can proceed with standard integration testing.

You now have a specification table, a risk register, and a scoping rationale — the three components of the Milestone 1 Proposal. Chapter 5 takes this artifact as its starting point: the specification defines the required behaviour at each subsystem boundary, and the risk register flags which interfaces carry the highest-consequence unknowns. Chapter 5 shows how to decompose the system into subsystems, define the interfaces between them, and produce the architecture diagram that makes integration testable.

CHAPTER 5

SYSTEM ARCHITECTURE & INTERFACE DEFINITION

5.0.0.0.1 Where this fits. You are now between Stage-Gate Stage 2 and Stage 3. Chapter 4 produced a specification and a risk register: you know *what* the prototype must do and which unknowns carry the highest risk — but not yet *how* to partition that specification into subsystems with verified boundaries. This chapter produces the next mandatory input to Gate 3 — the system architecture and interface table. Before you order a single component or write a single line of firmware, you must decompose your specification into subsystems and write down exactly what each subsystem promises to deliver to the next one. Architecture decisions made here are dramatically cheaper to change than the same decisions discovered at integration. A sound architecture is a prerequisite for an honest Gate 3 business case: if the blocks do not fit together on paper, they will not fit together on the bench.

By the end of this chapter you will be able to:

1. decompose a prototype specification into subsystems across the four domains: electronics, mechanical, firmware, and AI/IoT/robotic logic;
2. specify **interface contracts** that define the electrical, mechanical, data, and timing properties at each subsystem boundary;
3. allocate timing, power, and error budgets across subsystems and verify that the system total satisfies the specification; and
4. justify which subsystem boundary in a given architecture carries the highest integration risk by applying the detection and severity criteria from Chapter 4's FMEA framework.

5.1 Decomposition

Decomposition is the act of partitioning a system specification into a set of subsystems, each responsible for a bounded, well-defined portion of the total behaviour. Done well, decomposition gives every team member a clear scope and makes the integration task predictable. Done poorly — or skipped entirely — it produces

a monolithic prototype where every bug is a whole-system bug and no one is sure whose responsibility the failure is.

The discipline in this course groups prototype subsystems into four domains. Together they cover every boundary that a typical mechatronic, IoT, or robotic prototype will cross.

5.1.1 The Four Domains

5.1.1.1 Electronics

The **electronics subsystem** includes all hardware that converts physical quantities into digital signals and provides power conditioning for the rest of the system. In the running-example sorter this domain contains the item-detection sensor (time-of-flight or photoelectric), the camera or vision sensor feeding the AI pipeline, the microcontroller unit (MCU), motor drivers, and the power supply rail. Typical deliverables from this domain: a schematic, a net list, a power-rail diagram, and connector pinouts at every physical boundary.

5.1.1.2 Mechanical

The **mechanical subsystem** governs physical structure and motion: frame, conveyor belt, diverter actuator, mounting provisions for sensors and electronics, and the enclosure. In the prototype scope established in Chapter 2, the housing is functional-minimum — sufficient to hold the electronics securely and at the correct sensor geometry, but not production-finished. Mechanical deliverables: dimensional drawings for custom parts, assembly sequence, and the mechanical interface dimensions at every point where electronics or firmware interact with moving parts.

5.1.1.3 Firmware

The **firmware subsystem** is the embedded software running on the MCU. Its responsibilities are: reading the sensor, scheduling the AI inference call, issuing the actuator command, and publishing the MQTT status message. For the running-example sorter, the timing budget from Chapter 4 constrains this domain directly: the firmware must complete the entire sensor-to-actuator pipeline within 5 ms (established in Chapter 4 and formalised as Equation 5.1 in Section 5.3). Firmware deliverables: a task or interrupt map, a state-machine diagram, and a timing annotation for every I/O transaction.

5.1.1.4 AI/IoT/Robotic Logic

The **AI/IoT/robotic logic subsystem** encompasses the classification algorithm (the AI inference engine), the MQTT broker connection, and any cloud-side dashboard or data logging. This is the layer that gives the prototype its intelligence: the ability to distinguish item classes and to report status remotely. For the running example,

the central question from Chapter 3 — can the sub-$500 CAD BOM sorter achieve $\geq$95% sort accuracy at 6 items/min with MQTT status reporting? — is answered primarily by this domain, in concert with the electronics sensor quality. Deliverables: model architecture summary, inference-time benchmark, MQTT topic map, and QoS configuration.

5.1.2 Drawing the Block Diagram

The output of decomposition is a **system block diagram**: one box per subsystem, one labelled arrow per interaction. Figure 5.1 shows the block diagram for the running-example sorter. Every arrow in that figure will become one row in the interface table you will build by the end of this chapter.

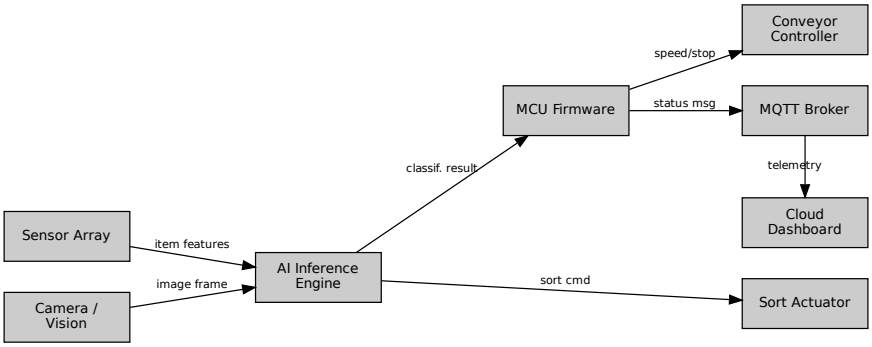

Figure 5.1: System block diagram for the IoT robotic sorting prototype. Boxes represent subsystems across the four domains; arrows represent interface boundaries, each of which receives a contract in Table 5.1. Labels A–D correspond to the four interface types introduced in Section 5.2.

5.1.2.0.1 In Practice. At the start of a team project, produce the block diagram before any other technical document. Pin it to the shared drive or project wall. Every team member should be able to point to their box and describe exactly what enters and exits it. A one-page interface table built from this diagram is the single most effective tool for keeping all team members building to the same specification. When everyone can see the same table, "I assumed it would be 3.3 V" is no longer a valid explanation for a three-day debugging sprint.

5.1.2.0.2 Common Pitfalls.

- **Starting the interesting subsystem before interfaces are fixed.** Firmware and AI engineers are often eager to write code before the electrical interfaces are settled. If the UART voltage level has not been agreed upon, any firmware written against an assumed level may be wrong, and the rework cost is real schedule time.
- **Treating the block diagram as decoration.** A block diagram that lives in a report appendix and is never referenced during build is not doing its job. It must be a living, binding contract — updated any time a subsystem boundary

changes, and consulted before any interface-crossing code or hardware is designed.

- **Verbal interface agreements.** "We'll figure out the data format later" is not an interface definition. The team that built the firmware and the team that built the AI classifier will make different assumptions, and those assumptions will collide at integration.

5.2 Interfaces as Contracts

An **interface** is the shared boundary between two subsystems. At that boundary one subsystem makes a promise to another. Calling it a **interface contract** is deliberate: both sides are obligated. The provider subsystem must deliver exactly what is specified; the consumer subsystem must not demand anything beyond what is specified. When the contract is written down, a violation is detectable. When the contract lives only in someone's head, it is invisible until the two subsystems meet on the bench.

There are four interface types used throughout this course. Each type has a different set of properties that must be specified to make the contract complete.

5.2.1 Electrical Interfaces

An **electrical interface** specifies:

- **Voltage level:** logic-high and logic-low thresholds (e.g., 3.3 V CMOS or 5 V TTL).
- **Current capacity:** the maximum source/sink current the provider can supply and the maximum the consumer may draw.
- **Protocol:** the communication standard (UART, SPI, I^2C, PWM, analog 0–5 V).
- **Connector:** physical connector type and pinout so that the mating is unambiguous.
- **Protection:** ESD handling, over-current limiting, or level-shifting requirements.

For the running-example sorter, the interface between the MCU and the motor driver is an electrical interface: 3.3 V PWM out from the MCU, 5 V logic input on the motor driver, requiring a level-shifter. Specifying the level-shifter requirement at architecture time — rather than discovering the incompatibility at bench time — is the direct return on investment for this chapter's work.

5.2.2 Mechanical Interfaces

A **mechanical interface** specifies:

- **Mounting geometry:** hole pattern, spacing, and tolerances.

- **Physical envelope:** maximum bounding box or keep-out volume.
- **Force or torque limits:** the maximum load the mating structure must carry.
- **Material compatibility:** relevant where corrosion, heat, or vibration is a factor.

For the running-example sorter, the mechanical interface between the sensor bracket and the conveyor frame defines the exact standoff height at which the time-of-flight sensor must sit relative to the belt surface. If the firmware team writes their detection algorithm assuming a 150 mm standoff but the mechanical team fabricates a 200 mm bracket, the item-detection gate geometry is wrong and every downstream timing calculation inherits that error.

5.2.3 Data Interfaces

A **data interface** specifies:

- **Protocol and transport:** MQTT, HTTP, I^2C register map, SPI frame format, serial byte stream.
- **Message format:** field names, data types, units, and byte order.
- **Topic or address:** MQTT topic string, I^2C device address, REST endpoint.
- **Quality of Service (QoS):** for MQTT, the QoS level (0, 1, or 2) and retained-message flag.
- **Error handling:** what happens if a message is lost, malformed, or arrives out of order.

For the running-example sorter, the data interface between the MCU firmware and the cloud dashboard is an MQTT publish on topic `sorter/status` with a JSON payload containing `item_class`, `sort_result`, and `timestamp_ms`. Any team member building the dashboard must consume exactly that format; any firmware update that renames a field without updating the interface table breaks the contract.

5.2.4 Timing Interfaces

A **timing interface** specifies:

- **Latency:** the maximum time between a trigger event and the expected response.
- **Throughput:** the maximum number of events per second the interface must sustain.
- **Jitter:** the maximum variation in latency from one cycle to the next.
- **Deadline type:** hard (a missed deadline is a system failure), firm (a missed deadline degrades output quality), or soft (a missed deadline is acceptable occasionally).

For the running-example sorter, the timing interface between the sensor read and the AI classifier is: sensor output available within $t_1 \leq 1$ ms of item detection, with jitter ≤ 0.1 ms. The AI classifier must begin its inference within that window to keep the 5 ms pipeline budget intact.

5.2.5 The Interface Table

All four interface types are collected into a single **interface table** — one row per arrow in the block diagram. Table 5.1 shows the interface table for the running-example sorter. This table is the primary deliverable of this chapter and must be completed before any subsystem build begins.

Table 5.1: Interface table for the IoT robotic sorting prototype. Each row names a boundary from Figure 5.1, its type, the contract properties, and the timing allocation from Section 5.3.

ID	From	To	Contract (key properties)	Latency
A	Sensor	Firmware	Electrical: 3.3 V I^2C, 400 kHz; range reading in 2-byte register; detection interrupt on GPIO	$t_1 \leq 1$ ms
B	Firmware	AI Engine	Data: shared memory frame buffer; 320×240 px RGB888; semaphore handshake	$t_2 \leq 2$ ms
C	AI Engine	Firmware	Data: classification label (uint8) + confidence (float32); written to result register; sets done flag	included in t_2
D	Firmware	Motor Driver	Electrical: 3.3 V PWM → level-shifted 5 V; direction GPIO; 20 kHz carrier	$t_3 \leq 2$ ms
E	Firmware	MQTT Broker	Data: TCP/IP; topic `sorter/status`; JSON payload; QoS 1; ≤200 ms publish latency	async
F	Sensor Bracket	Conveyor Frame	Mechanical: M3 × 4-hole pattern, 20 mm spacing; standoff 150 ± 2 mm; max load 0.5 N	N/A

5.3 Budget Allocation

Specifying interfaces is necessary but not sufficient. You must also verify that the sum of all subsystem demands does not exceed the system-level constraint. This verification is called **budget allocation**: you divide a finite resource — time, power, or permissible error — across the subsystems that consume it, and then confirm the total stays within the specification limit.

Three budgets matter for the running-example prototype: timing, power, and measurement error.

5.3.1 Timing Budget

The **timing budget** formalises the constraint that the sum of all pipeline-stage latencies must not exceed the specification deadline.

$$T_{\text{total}} = \sum_{i} t_i \leq T_{\text{budget}} \tag{5.1}$$

This is the budget structure introduced in Chapter 4's risk analysis (Equation 4.3). Here it is stated formally as the governing inequality for the architecture. Every timing interface in the interface table must carry a t_i allocation; those allocations are substituted into Equation 5.1 to verify the total.

5.3.2 Power Budget

The **power budget** verifies that the supply rail can deliver enough current for all subsystems simultaneously.

$$P_{\text{total}} = \sum_{i} P_i \tag{5.2}$$

P_{total} must not exceed the rated output of the power supply. For the running-example sorter on a 5 V/2 A USB supply (10 W), typical subsystem allocations are: MCU 0.5 W, camera module 1.2 W, motor driver (idle) 0.8 W, motor driver (active) 2.5 W, Wi-Fi module 0.9 W — a peak total of 5.9 W, well within the 10 W budget with a 4.1 W margin. Margins matter: a motor driver that draws more than rated, a Wi-Fi transmit burst that spikes current, or a future addition of an LED indicator can each erode headroom. Figure 5.2 shows the timing and power budgets as stacked bars against their respective limit lines.

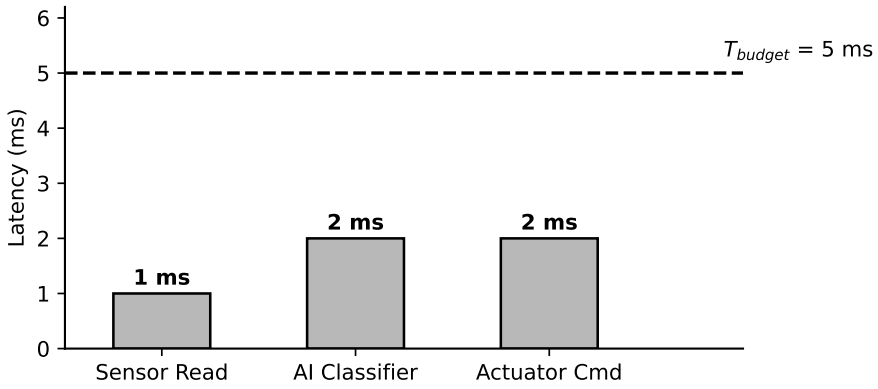

Figure 5.2: Interface budgets for the running-example sorter. Left panel: timing budget — stacked bar of t_1, t_2, t_3 against the 5 ms limit line (Equation 5.1). Right panel: power budget — stacked bar of subsystem power draws against the 10 W supply limit (Equation 5.2). Zero margin on timing means any added pipeline step requires removing an existing one.

5.3.3 Error Budget

Multiple measurement stages each contribute their own uncertainty. Assuming the error sources are independent and random, they combine in **quadrature** (root-sum-of-squares, RSS):

$$e_{\text{total}} = \sqrt{\sum_i e_i^2} \tag{5.3}$$

The RSS model is more realistic than a worst-case linear sum because independent errors partially cancel. If e_{total} exceeds the acceptance criterion, the team must tighten the largest individual contributor first — squeezing a small contributor yields almost no benefit once a dominant error source exists.

5.3.4 Worked Example — Part A: Timing Budget

The central question from Chapter 3 requires classifying and diverting each item within $\leq$167 ms of arrival (6 items/min), but the firmware pipeline latency must be far shorter so that the MQTT publish and mechanical diversion can complete within that window. Chapter 4's risk analysis established a firmware timing budget of $T_{\text{budget}} = 5$ ms with three pipeline stages. Applying Equation 5.1:

$$T_{\text{total}} = t_1 + t_2 + t_3 = 1 \text{ ms} + 2 \text{ ms} + 2 \text{ ms} = 5 \text{ ms}$$

$$T_{\text{total}} = 5 \text{ ms} \leq T_{\text{budget}} = 5 \text{ ms} \quad \checkmark$$

where:

- $t_1 = 1$ ms: sensor read (I^2C transaction, distance measurement, interrupt service);
- $t_2 = 2$ ms: AI classifier inference (image capture, pre-processing, forward pass, label decode);
- $t_3 = 2$ ms: actuator command (PWM update, GPIO direction set, motor driver settling).

The budget passes, but the **dwell-time margin is zero**. Any added pipeline step — a second sensor read for confirmation, a logging write to flash, a display update — requires shortening one of the existing stages by an equal amount. This constraint must be visible to every team member: firmware engineers do not add steps to this pipeline without a corresponding timing analysis showing where the recovered time comes from.

5.3.5 Worked Example — Part B: RSS Error Budget

The sort-accuracy specification requires $\geq$95% classification accuracy. Three independent sources of measurement uncertainty contribute to positional error at the classification gate:

- $e_1 = \pm 0.05$ m/s: conveyor speed sensor uncertainty, which introduces positional error when the system computes item position from belt velocity. At the

nominal belt speed of 0.05 m/s, a speed uncertainty of ± 0.05 m/s maps to a relative positional uncertainty of $\pm(0.05/0.05) \times 1\% = \pm 0.1\%$; this is the value substituted as e_1 in the RSS calculation.

- $e_2 = \pm 0.5\%$: AI vision classification uncertainty (probability gap between top-1 and top-2 class, expressed as a percentage-point uncertainty in the sort decision).
- $e_3 = \pm 0.3\%$: positional uncertainty equivalent derived from timing jitter of ± 0.1 ms mapped through the nominal belt speed.

Expressing all contributors in compatible units (percentage-point equivalent positional uncertainty) and applying Equation 5.3:

$e_1 = 0.1\%$, $e_2 = 0.5\%$, $e_3 = 0.3\%$

$$e_{\text{total}} = \sqrt{(0.1\%)^2 + (0.5\%)^2 + (0.3\%)^2} = \sqrt{0.01 + 0.25 + 0.09}\% = \sqrt{0.35}\% \approx 0.59\%$$

The acceptance criterion is $e_{\text{total}} \leq 0.6\%$.

$$e_{\text{total}} \approx 0.59\% \leq 0.60\% \quad \checkmark$$

The error budget passes — but barely. The dominant contributor is e_2, the AI classification uncertainty. If the model's confidence gap narrows (e.g., the item class becomes harder to distinguish), this budget will fail at that contributor first. Chapter 6 will revisit this when selecting the camera sensor and AI model architecture.

5.3.5.0.1 In Practice. A team building a motor controller prototype skipped the electrical interface table and started firmware before agreeing on the UART voltage level between the MCU and the motor driver. The MCU operated at 3.3 V logic; the motor driver expected 5 V logic. The firmware team wrote and tested their UART code against a bench logic analyser at 3.3 V. The electronics team wired the motor driver directly to the MCU TX pin. At integration, the motor driver received marginal logic levels: it sometimes responded and sometimes did not, depending on supply current at the moment of the transaction. Diagnosing the intermittent fault cost three days. The fix was a single-chip level-shifter — a $0.30 CAD part that would have taken ten minutes to add to the schematic during architecture definition. The interface table entry for that boundary would have read: *"Electrical: MCU UART TX at 3.3 V CMOS logic; motor driver UART RX requires 5 V TTL; level-shifter required on this net."* Three days of debugging for a $0.30 part and one table row.

5.4 Why Integration Fails at Boundaries

Subsystems fail at their boundaries, not at their centres. This statement is almost universally true in mechatronic and embedded prototype development, and it has a straightforward explanation.

Within a subsystem, one engineer (or one small team) controls all the assumptions. The firmware engineer who writes the sensor driver and the interrupt handler is working within a single, consistent mental model. The mechanical engineer who designs the conveyor frame is making choices within their own constraints. Bugs within a subsystem are caught during subsystem-level testing because the developer is testing exactly the thing they understand.

At a boundary, two or more parties each independently assumed something about the other's subsystem. Those assumptions were never formally compared. When the two subsystems meet for the first time, every mismatched assumption becomes a defect. Figure 5.3 illustrates a canonical boundary mismatch: two subsystems whose interface contracts were never formally aligned, resulting in an incompatible handshake at integration.

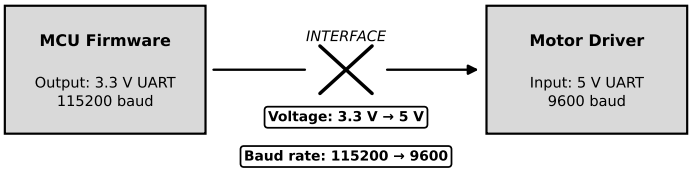

Figure 5.3: Boundary mismatch at integration. The left subsystem (MCU firmware) assumes 3.3 V CMOS logic and outputs a valid signal by its own standard; the right subsystem (motor driver) expects 5 V TTL and interprets the same signal as an indeterminate logic level. Neither subsystem contains a bug in isolation; the defect exists only at the boundary, and is invisible until the two are connected.

5.4.1 Taxonomy of Boundary Failures

Four categories of boundary failure recur across student and professional projects alike.

5.4.1.1 Voltage Level Mismatch

The most common electrical boundary failure: one subsystem drives 3.3 V logic, the adjacent subsystem expects 5 V, and neither team checked the other's datasheet. Outcome: intermittent communication, latent damage to 3.3 V pins exposed to 5 V, or total non-function. Prevention: one row in the electrical interface table, one level-shifter on the BOM.

5.4.1.2 Protocol Mismatch

One subsystem writes a SPI frame formatted as 16-bit big-endian; the consuming subsystem reads it as 16-bit little-endian. Both subsystems pass their individual unit tests. The integration test produces garbage output that takes hours to trace to a byte-order disagreement. Prevention: specify byte order, data type, and endianness in the data interface contract.

5.4.1.3 Timing Assumption Mismatch

The firmware team assumes the AI engine returns a result within 2 ms. The AI team benchmarks their model at 3 ms on the prototype hardware. The pipeline budget fails by 1 ms and the diverter fires 1 ms after the item has already passed the gate. Prevention: the timing interface contract specifies the maximum latency; the AI team runs a benchmark on the actual target hardware and signs off before build begins.

5.4.1.4 Mechanical Fit Mismatch

The sensor bracket is designed for a sensor with an M2.5 mounting hole. The sensor that arrives has an M3 mounting hole. Fabrication is already complete. Prevention: the mechanical interface contract specifies the exact hole pattern, fastener size, and tolerance, and is verified against the component datasheet before fabrication is released.

5.4.2 Identifying the Highest-Risk Boundary

Not all boundaries carry equal risk. The boundary with the highest integration risk is the one where:

1. the two subsystems are developed by different people or teams with limited communication;
2. the interface contract is the least precisely specified; and
3. a mismatch at this boundary would be the hardest to detect before it causes visible system failure.

This is precisely the FMEA detection factor D from Chapter 4 applied at the interface level. For the running-example sorter, the firmware-to-AI-engine boundary (Interface B in Table 5.1) carries the highest integration risk: the shared memory frame buffer protocol is non-standard, the two subsystems may be developed on different toolchains, and a timing assumption mismatch here propagates directly into a missed divert-gate deadline.

5.4.2.0.1 Common Pitfalls.

- **"We'll figure it out later" interface agreements.** Verbal agreements about data formats, voltage levels, or timing are not interface contracts. They are deferred arguments. Experienced engineers know that "later" always arrives during integration, when schedule pressure is highest and the cost of rework is greatest.
- **Developing the interesting subsystem first.** AI engineers naturally want to train and test the classifier. Firmware engineers want to write drivers. Without fixed interfaces, each group builds to their own internally consistent assumptions,

and integration becomes a negotiation between two finished products that were never designed to meet each other.

- **Interface table as a compliance artifact.** Filling out the interface table to satisfy a deliverable requirement, then ignoring it during build, defeats its purpose. The table must be the reference document that every team member checks before crossing a subsystem boundary.

Try It Yourself

Scenario: You are architecting a wireless air-quality monitor. The specification requires a total firmware pipeline latency of $T_{\text{budget}} = 8$ ms. Three pipeline stages have been proposed:

- $t_1 = 2$ ms: ADC sample (sensor read and digitisation);
- $t_2 = 3$ ms: digital filter (moving-average computation on the MCU);
- $t_3 = 2$ ms: BLE transmit buffer (packetise reading and push to BLE stack).

Three measurement error sources contribute to the reported air-quality reading:

- $e_1 = \pm 0.3$°C equivalent: temperature sensor uncertainty;
- $e_2 = \pm 1.5$°C equivalent: humidity sensor uncertainty (the % RH specification has been pre-converted to a temperature-equivalent for this budget);
- $e_3 = \pm 0.1$°C equivalent: ADC quantization noise.

The acceptance criterion for the combined error is $e_{\text{total}} \leq 1.6$°C equivalent.

Tasks:

(a) **Timing budget.** Using Equation 5.1, compute $T_{\text{total}} = t_1 + t_2 + t_3$. State whether the timing budget is satisfied and identify the remaining margin (if any).

(b) **Error budget.** Using Equation 5.3, compute e_{total} from the three error sources. State whether the error budget passes the acceptance criterion and identify the dominant contributor.

(c) **Constraint violation response.** If the firmware team's digital filter implementation turns out to require $t_2 = 4$ ms instead of 3 ms, T_{total} will exceed T_{budget}. Identify which stage(s) could be shortened to restore compliance. For each option you propose, state the new t_i value and verify that the revised $T_{\text{total}} \leq 8$ ms.

(d) **Interface contract.** Identify one interface in this air-quality monitor system (choose a boundary you consider the highest integration risk). Write a two-line interface contract for that boundary, specifying: (line 1) the protocol and voltage level or data format; (line 2) the timing constraint.

Review Questions

1. **(Conceptual)** State the four interface types defined in this chapter. For each type, give one specific example from the running-example IoT sorter system, naming the two subsystems at the boundary and one key property of the contract.

2. **(Conceptual)** Explain in two sentences why integration bugs typically appear at boundaries between subsystems rather than within a subsystem. Your answer should reference the concept of locally consistent assumptions within a subsystem versus the unverified assumptions that cross a boundary.

3. **(Conceptual — classification)** A team proposes an interface between the MCU and a Bluetooth module described only as "send the JSON string over serial." Classify the completeness of this interface contract: state which of the four interface types it partially addresses, which required properties are missing, and explain why the omissions constitute integration risk.

4. **(Applied — calculation)** A firmware pipeline has four stages with the following latency allocations: $t_1 = 0.8$ ms, $t_2 = 1.2$ ms, $t_3 = 0.6$ ms, $t_4 = 0.9$ ms. The timing specification requires $T_{\text{budget}} = 4$ ms.

   (a) Compute T_{total} using Equation 5.1.

   (b) State whether the timing budget is satisfied. If it is satisfied, state the margin.

   (c) An added 0.8 ms logging step is required. How must the existing stage allocations be adjusted to keep $T_{\text{total}} \leq 4$ ms? State which stage(s) to shorten and by how much.

5. **(Applied — multi-step)** Three subsystems each contribute independent positional measurement error to a parts-placement robot: encoder $e_1 = \pm 0.2$ mm, vision sensor $e_2 = \pm 0.5$ mm, synchronisation jitter equivalent $e_3 = \pm 0.1$ mm. The acceptance criterion is $e_{\text{total}} \leq 0.7$ mm.

   (a) Compute e_{total} using Equation 5.3. Show all intermediate steps.

   (b) State whether the error budget passes the acceptance criterion. If it passes, state the margin; if it fails, state the deficit.

   (c) The vision sensor is upgraded to a model with ± 0.3 mm uncertainty. Recompute e_{total} and state the new margin against the 0.7 mm criterion.

End-of-Chapter Artifact

By the end of this chapter your project folder must contain one deliverable:

System Architecture Package: the system block diagram (Figure 5.1) plus a completed interface table with every subsystem boundary from that diagram named, its four-property contract defined, and the timing and power budget columns filled in and verified against Equations 5.1 and 5.2.

Specifically, the package must demonstrate:

- A labelled block diagram with one box per subsystem across the four domains (electronics, mechanical, firmware, AI/IoT/robotic logic) and one labelled arrow per interface.
- An interface table in which each row names the two subsystems at the boundary, states the interface type (electrical, mechanical, data, or timing), and specifies the key contract properties.
- A timing budget column showing t_i allocations that sum to $T_{\text{total}} \leq T_{\text{budget}}$ (Equation 5.1).
- A power budget column showing P_i allocations that sum to P_{total} (Equation 5.2), with the total compared to the rated supply output.
- A written identification of the highest-risk boundary, with a two-sentence justification referencing the interface type and the detection difficulty of a mismatch at that boundary.

The block diagram and completed interface table together form the **Gate 3 architecture deliverable** (see the Stage-Gate model established in Chapter 2). A Gate 3 reviewer will verify: (1) every arrow in the block diagram has a corresponding row in the interface table; (2) each row carries a complete four-property contract; and (3) the timing and power budget columns close against Equations 5.1 and 5.2. A package that fails any of these three checks does not satisfy the Gate 3 architecture criterion.

Chapter Summary

Three tools now equip you to move from specification to buildable architecture.

First, a **decomposition framework**: dividing any prototype into four subsystem domains — electronics, mechanical, firmware, and AI/IoT/robotic logic — gives every team member an unambiguous scope and transforms integration from a surprise into a managed handshake.

Second, a **contract language for interfaces**: every boundary between subsystems carries a formal contract specifying electrical properties, mechanical geometry, data format, and timing constraints. A boundary without a contract is a liability that will be paid at integration time. The interface table is the instrument that makes all contracts visible to the whole team simultaneously.

Third, a **budget discipline**: timing, power, and measurement error are finite resources. Equations 5.1, 5.2, and 5.3 provide the arithmetic to verify that the sum of subsystem demands fits within the system-level limits. For the running-example sorter, the 5 ms timing budget is exactly consumed by the three pipeline stages, leaving zero margin; the power budget carries a 4.1 W margin; and the RSS error budget passes the 0.6% criterion by the narrowest of margins, flagging AI classification uncertainty as the risk to watch.

The architecture established here is the foundation on which Chapter 6 builds: the System Architecture Package gives Chapter 6 its starting point, and every component selected there will be evaluated against the interface contracts written here.

J. F. N. Salik,

INDEX